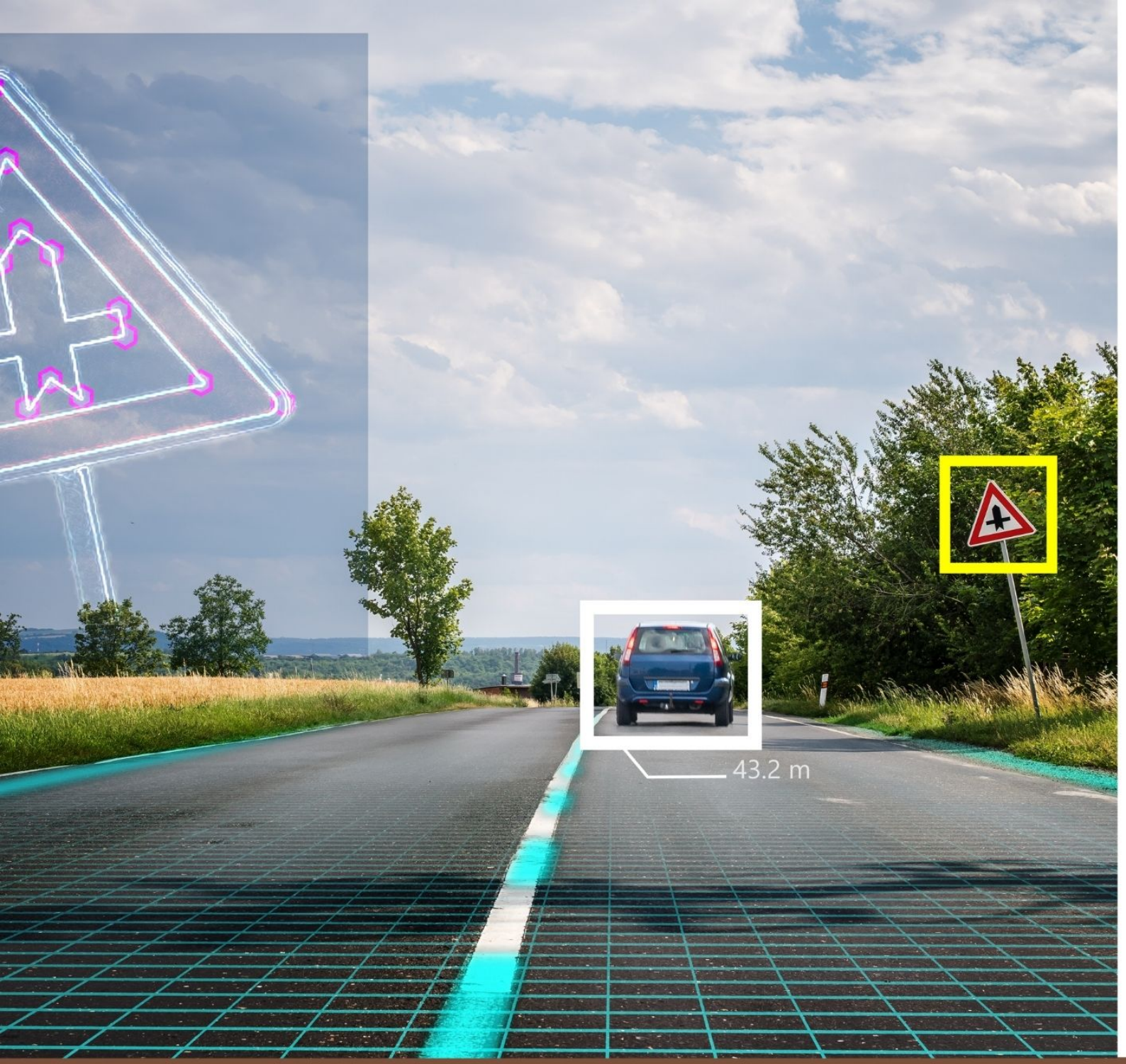


COMPUTER VISION

Prof. Jayanta Mukhopadhyay
Computer Science and Engineering
IIT Kharagpur

INDEX

S. No	Topic	Page No.
	Week 1	
1	Fundamentals of Image Processing Part I	1
2	Fundamentals of Imagr Processing Part II	21
3	Image Transform Part I	39
4	Image Transform Part II	58
	Week 2	
5	Projective Geometry â€“ Part I	79
6	Projective Geometry â€“ Part Ii	94
7	Projective Transformation	114
8	Homography:Properties â€“ Part I	140
9	Homography:Properties â€“ Part Ii	155
10	Homography:Properties â€“ Part Iii	172
	Week 3	
11	Camera Geometry â€“ Part I	189
12	Camera Geometry â€“ Part Ii	208
13	Camera Geometry â€“ Part Iii	223
14	Camera Geometry â€“ Part Iv	240
15	Camera Geometry â€“ Part V	261
	Week 4	
16	Stereo Geometry â€“ Part I	279
17	Stereo Geometry â€“ Part Ii	295
18	Stereo Geometry â€“ Part Iii	310
19	Stereo Geometry â€“ Part Iv	322
	Week 5	
20	Stereo Geometry â€“ Part Vi	334
21	Stereo Geometry â€“ Part V	346
22	Stereo Geometry â€“ Part Vii	363
23	Stereo Geometry â€“ Part Viii	380
	Week 6	
24	Feature Detection And Description â€“ Part I	394
25	Feature Detection And Description â€“ Part II	413
26	Feature Detection And Description â€“ Part III	432
27	Feature Detection And Description â€“ Part IV	449

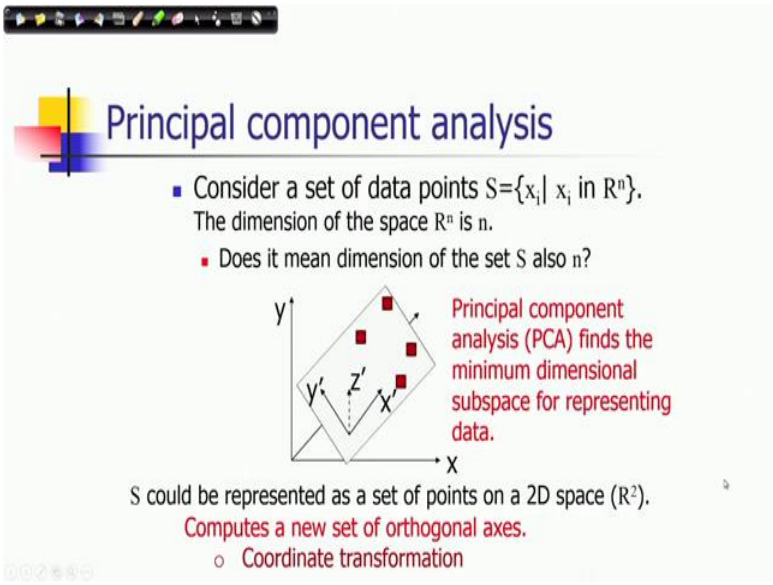
28	Feature Detection And Description â€“ Part V	474
	Week 7	
29	Feature Matching And Model Fitting- Part I	485
30	Feature Matching And Model Fitting- Part Ii	502
31	Feature Matching And Model Fitting- Part Iii	514
32	Feature Matching And Model Fitting- Part Iv	527
33	Feature Matching And Model Fitting- Part V	540
	Week 8	
34	Color Fundamentals And Processing-Part I	558
35	Color Fundamentals And Processing-Part Ii	574
36	Color Fundamentals And Processing-Part Iii	591
37	Color Fundamentals And Processing-Part Iv	613
38	Color Fundamentals And Processing-Part V	633
39	Color Fundamentals And Processing-Part Vi	651
40	Color Fundamentals And Processing-Part Vii	665
	Week 9	
41	Range Image Processing â€“ Part I	679
42	Range Image Processing â€“ Part Ii	693
43	Range Image Processing â€“ Part Iii	707
44	Range Image Processing â€“ Part IV	720
45	Range Image Processing â€“ Part V	735
	Week 10	
46	Clustering And Classification Â€“ Part I	757
47	Clustering And Classification Â€“ Part Ii	780
48	Clustering And Classification Â€“ Part Iii	799
49	Clustering And Classification Â€“ Part Iv	816
50	Clustering And Classification Â€“ Part V	832
	Week 11	
51	Dimensional Reduction And Sparse Representation â€“ Part	855
52	Dimensional Reduction And Sparse Representation â€“ Part	871
53	Dimensional Reduction And Sparse Representation â€“ Part	891
54	Dimensional Reduction And Sparse Representation â€“ Part	909
	Week 12	
55	Deep Neural Architecture And Applications â€“ Part I	927
56	Deep Neural Architecture And Applications â€“ Part II	950
57	Deep Neural Architecture And Applications â€“ Part III	970

58	Deep Neural Architecture And Applications â€“ Part IV	988
59	Deep Neural Architecture And Applications â€“ Part V	1007
60	Deep Neural Architecture And Applications â€“ Part VI	1028

Computer Vision
Prof. Jayanta Mukhopadhyay
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

Lecture - 51
Dimension Reduction and Sparse Representation Part - 1

(Refer Slide Time: 00:24)



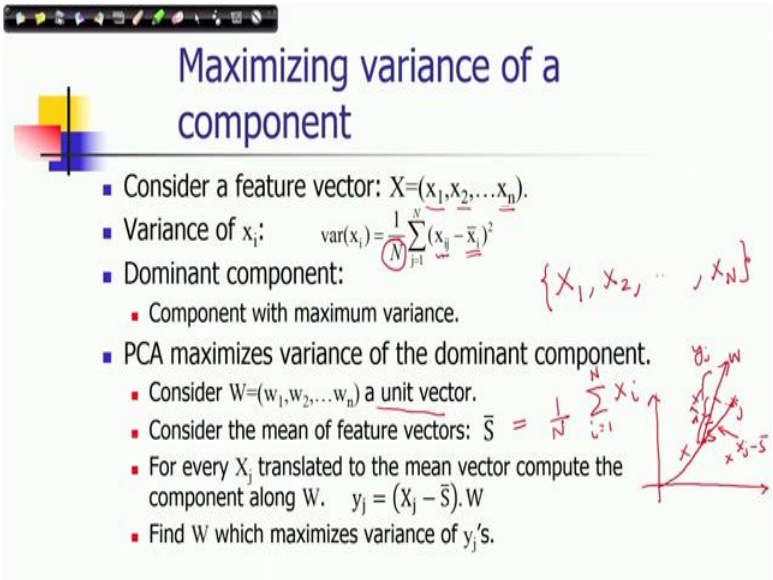
In this lecture, we will discuss about Dimension Reduction and Sparse Representation. So, let us first understand that, what is meant by dimension of a data? Consider a set of data points as shown in the slide that it is a set, where xi is a data point in the space of N dimensional real space. We can consider it is a vector in that real space so, the dimension of that space is naturally n.

Now does it mean the dimension of the set S is also n? So, let us take this example. So, it is a 3 dimensional space and there are 4 data points. Now, they may be arranged in such a way that they lie on a plane. So, when they lie on a plane we can always define a coordinate system within that plane and use that coordinate convention to represent every point.

So, in that case all the points could be represented as a set of points on it 2D space or which is a 2D real space. So, it is not necessary that dimension of data would be the same as a dimension of the space, it could be lower than that number, what the example has been shown here. So, the principal component analysis is a method by which it finds the minimum dimensional subspace for representing data. So, we will learn in this lecture how

this analysis could be done and the basic idea here is that, it computes a new set of orthogonal axis. And that and using that a new set of orthogonal axis you define new coordinates with respect to the representation.

(Refer Slide Time: 02:34)



So, it is a coordinate transformation in one sense. What principal component analysis does? It maximizes variance of a component let me explain that: you consider a feature vector representation of that data point X and since it is represented in n dimensional space. So, we have n components or n fields of the vector, which is shown here as {x1, x2,..,xn}; this is a convention we are using for representing a data.

Now variance of a particular component say ith component xi that is defined as using this mathematical expression, this should be clear that if there are N such data points; that means, N data points. So, for any xi's vector the corresponding jth data point we considered that is a value for the jth component of xi's vector.

And you consider the mean of that component. So, this is how the variance is defined, the standard definition of variance. Now we say a component is dominant out of all these n components whose variance is the maximum. So, that is then we say that component is a dominant component. Now, PCA it maximizes the variance of the dominant component. Let us understand what it means, consider a unit vector W and since as I mentioned that there is a coordinate transformation involved in PCA what we can do, that we can perform

certain type of coordinate combination, conventions. So, center of the coordinate our origin of the coordinate can be considered as a mean of these feature vectors.

So, let us represent that mean by this $\bar{S}$, which means you have N feature vector saved this is {X1, X2,..,XN} N feature vectors and $\bar{S}$ should be

$$\bar{S} = \frac{1}{N}\sum_{i=1}^{N} X_i$$

So, this is how the mean could be computed. So, for every Xj translate it to the mean vector compute the component along W. Suppose you can consider these are the data points and this is $\bar{S}$. So, we are translating and this is the original coordinate of the sparse. So, first we transfer the center to $\bar{S}$ take any particular direction, say this is the direction W and take the component means the dot product of this W.

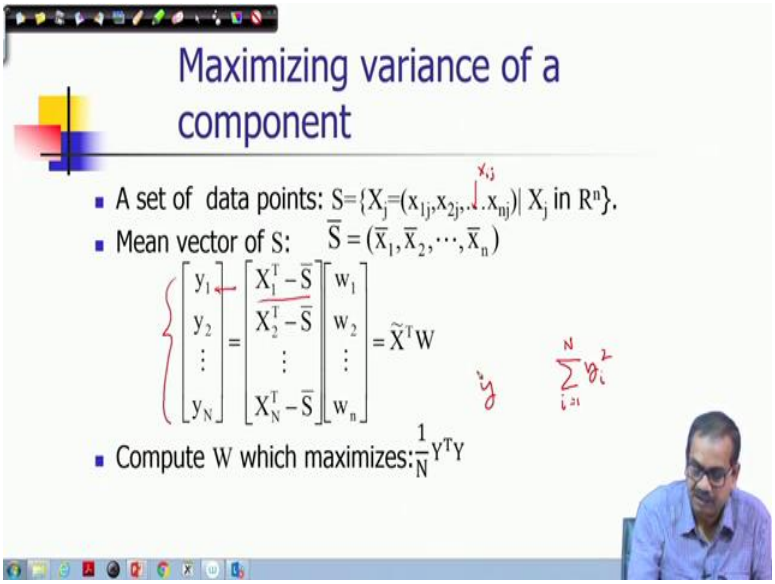
This is a unit vector, this should be a unit vector and so, the dot product of this, which would be the component that is how you will be computing it. So, this is what is yj.

$$y_j = (X_j - \bar{S}).W$$

So, you take which is a unit vector in this case, we consider this is a unit vector and then you take the dot product of this then you get this component as yj.

Now you consider the variance of this component. So, the problem of PCA is at least you have to find out one such direction, where it maximizes the variances of these projections of different vectors all the data a point centering at its mean.

(Refer Slide Time: 07:05)



So, let me continue that with that representation, we have a set of data points and now we are representing the data points in this way that, if I have Xj as a vector then, for the jth vector there are small n number of components. And, each one component is a variable which is indexed as the ith component should be Xij that is are how we are representing it. So, we consider ith component as Xij here.

So, the mean vector as we discussed is $\bar{S}$which means this is a mean of each component we have defined it earlier also. And, then we perform this transformation we are taking the component along a vector W which is a unit vector and you can see that every vector is translated towards mean. And, for all N vectors there are N numbers. So, for every vector translated it and we have the corresponding component along the unit vector.

So, finally, you get N observations or projections of all these data points along W centering at the mean of the vectors. So, now you consider the variances of these y's which can be represented as. So, we can write compute W which maximizes

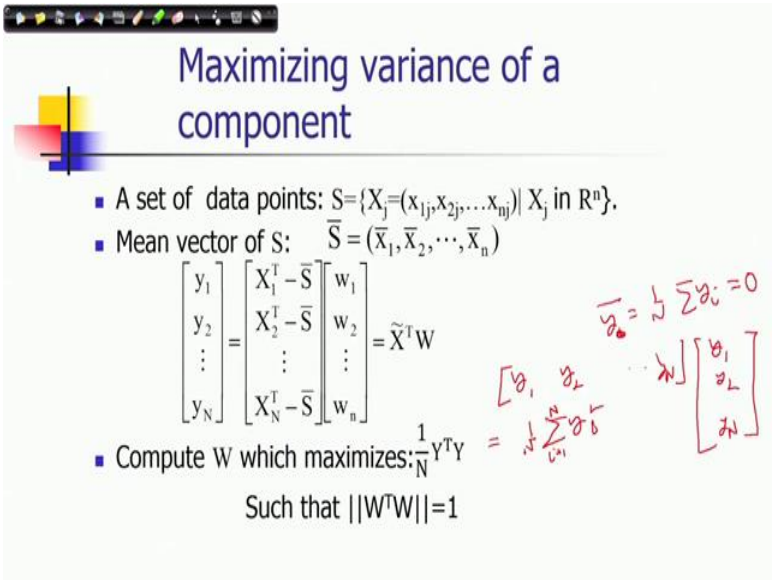
$$\frac{1}{N} Y^T Y$$

Actually you can say that since we have translate towards mean. So, the mean of the y's would be 0.

So, it is sufficient if I simply maximize the square of the magnitudes of that vector.

$$\left|\left|W^T W\right|\right| = 1$$

(Refer Slide Time: 09:47)



So, these are magnitude of this vector that would be

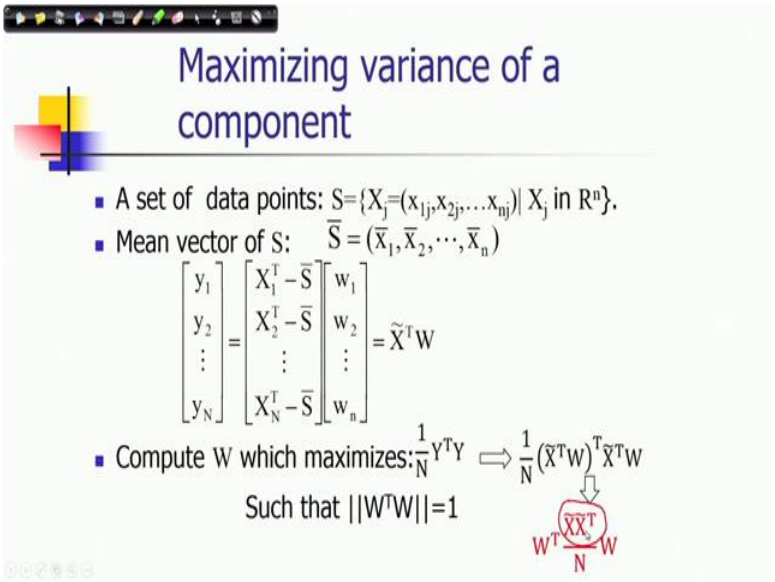
$$\frac{1}{N}\sum_{i=1}^{N} y_i^2$$

And we are taking the mean of it, which is the variance of this value. And, if I consider the mean of these mean of the y 1's at all we can write

$$\bar{y} = \frac{1}{N}\sum_{i=1}^{N} y_i \ = 0$$

And effectively we are finding out the variance of the components and we would like to get W which maximizes this particular factor. There is a constraint on W you have already mentioned that it has to a unit vector so we should have the norm of the W that should be equal to 1.

(Refer Slide Time: 10:50)

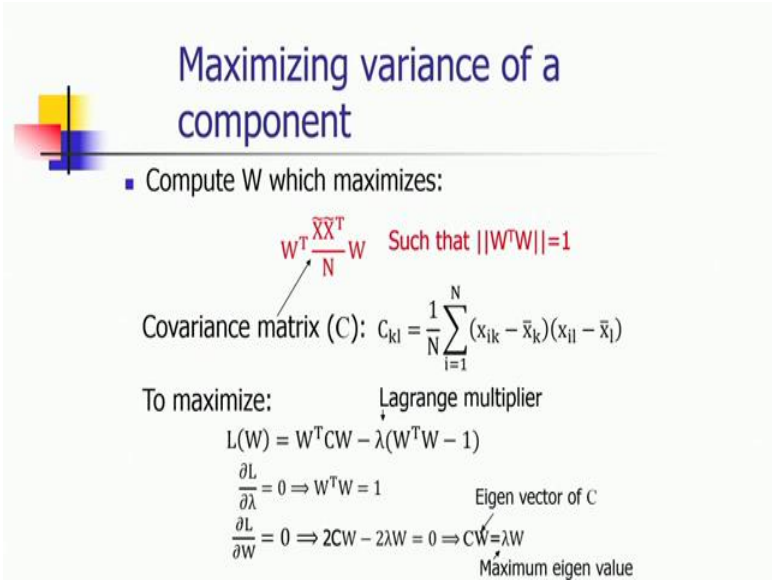


And, effectively if I consider expand Y^T or Y, y is it has been shown it has been X^TW. So, this can be written in this form and which is giving the expression as

$$\frac{1}{N}(\widetilde{X^T}W)^T\widetilde{X^T}W$$

This quantity is interesting because, what it is measuring? It is measuring the covariance of X of course, you have to consider the averaging of that those values. So, this is the averaging there is a term which is divided by N which will average it out.

(Refer Slide Time: 11:36)



So, we have to compute W which maximizes this particular factor and as I mentioned that this is nothing, but the covariance matrix C, where the element of say kth lth element is covariance between the kth component and lth component. Covariance between kth component and lth component of the vectors, of the data points and that is how the covariance matrix is defined.

So, the objective function you can consider for maximizing the variance is that to maximize a function, which is the function of weight vector W or the unit directions W. And, which will maximizing this quantity; this is the quantity which is same as this one which has to be maximized, but there is a constraint so, this constraint in the objective function can be incorporated using Lagrange multiplier. And this is that particular term. So, this lambda is the Lagrange multiplier.

So, if I take the derivative with respect to lambda it will be $W^TW=1$ which is actually enforcing the constraint what we want to for while getting a solution. And, then if I take the partial derivative with respect to the weight direction unit vector W, then we get this system of equations as that I earlier also mentioned that the similar properties or similar rules of differential geometry for one dimensional variable can be extended for multi dimensional space also when you are using matrix operations, linear operations.

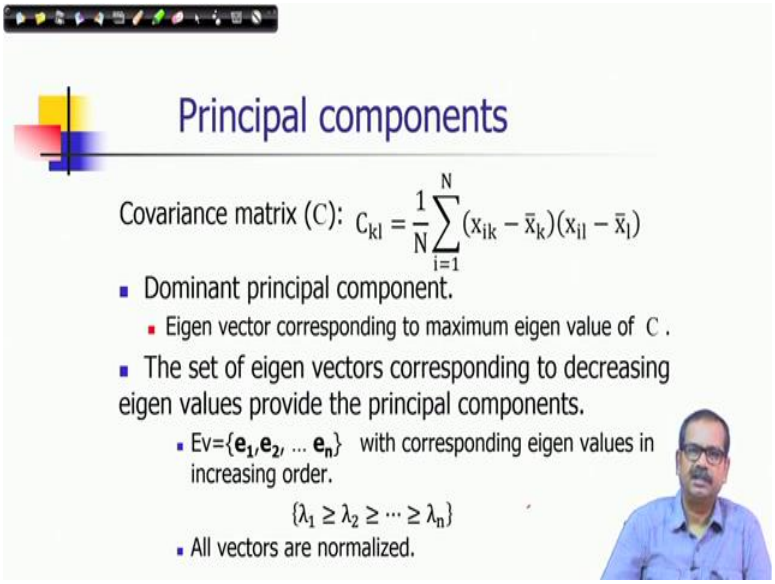
So, you can consider W^TCW it is a quadratic product.

$$L(W) = W^T CW - \lambda(W^T W - 1)$$

$$\frac{\partial L}{\partial W} = 0 \rightarrow 2CW - 2\lambda W = 0 \rightarrow CW = \lambda W$$

So, λ is an eigen value and since we are considering W has to be an unit vector. So, you will consider the unit eigenvector in this case. And, since we would like to maximize the variances and we will be considering the maximum eigen value for the vector corresponding to the maximum eigen value.

(Refer Slide Time: 14:47)

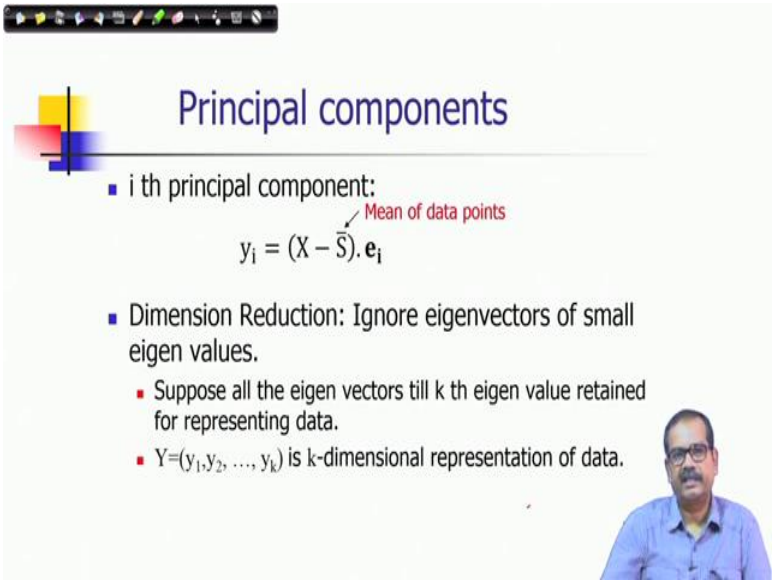


So, what you get actually dominant principal component that is eigenvector corresponding to maximum eigen value of C. Now, what about other eigenvectors because as covariance matrix it is a symmetric matrix so, you will get N eigenvectors if they because a dimension is small N here in this case and n X n matrix.

And so, all the eigenvectors in fact, it can be shown that they are providing the maximum variances alone the residuals one after another. And, so the solution of this particular analysis or what you can say that set of eigenvectors corresponding to decreasing eigen values they provide the principal components.

Suppose we represent it set in this form that there are small in number of eigenvectors as I say {e1, e2,..,en} such that there are corresponding eigen values are also in the decreasing order. So, e1 corresponds to the maximum eigen value, e2 corresponds to the second maximum and the minimum eigen value λn that corresponds to the eigenvector en. You should note all vectors are normalized here, we are only considering unit vector eigen vector.

(Refer Slide Time: 16:09)



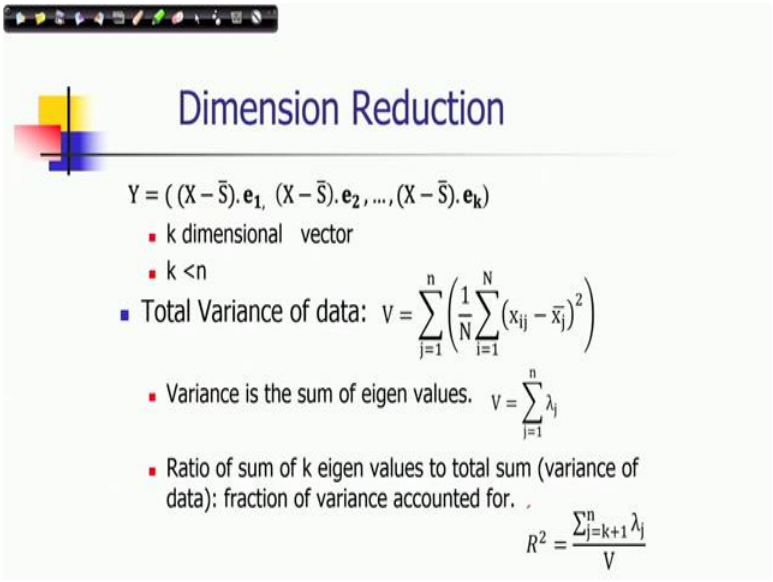
So, the ith principal component is defined as the projection along the eigen vector of the centering at $\bar{S}$ centering at the mean of the data points. So, we can mathematically write it as

$$y_j = (X - \bar{S}).e_i$$

The dimension reduction could be in this way, that we can ignore eigenvectors of small eigen values. That means, for a data point we can reduce the dimension, we can ignore those components whose eigen values are very less.

So, there is an interpretation of eigen values, they are representing the variances of the residuals for at that point. So, suppose all the eigen vector still kth eigen value retained for representing data, then the data can be approximately represented by k dimensional, it kind of a k dimensional representation of data.

(Refer Slide Time: 17:09)



So, we had an N dimensional data, but as I mentioned the dimension of data is not necessarily the dimension of the space. So, they can lie on a subspace which is a k dimensional subspace in this case. And using the principal component analysis we have performed that coordinate transformations. So now, your coordinate axis are given by these eigen vectors and you are considering and your center of your coordinates becomes the center of the data point. And, you consider the projections of data points along each eigen vectors that would give you the components.

So, first k components in decreasing order which are sufficient which may be sufficient to represent the data, which may be sufficient to capture the variances of the data. So, this is a thing that we have k dimensional vector and as you understand k has to be less than n or it could be equal n also. So, total variance of data that can be you know, that can be represented as variances of each component sum of variances of each component.

There are n components so, you consider variance of each component and that would give you the total variance of data. So, variance is the sum of eigen values. So, this can be shown, this can be proved also, that variance is nothing, but sum of eigen values. So, that is why the eigen value which are very small which is negligible it is not contributing to the data variance part and we can ignore those components. So, ratio of sum of k eigen values to total sum that is a variance of the data that is the fraction of variance accounted for. So,

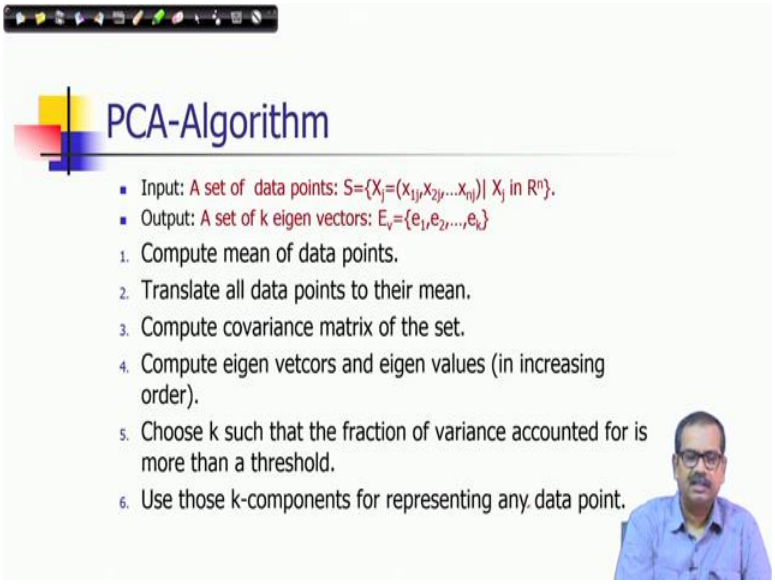
in dimension reduction this is what we would be considering that as high as this fraction is better is the information content of the data retained.

So, this fraction is a fraction as you understand it varies from 0 to 1. So, we will be considering a very high value of this fraction nearly 1 for representing data. So, mathematically we represent this statistics as

$$R^2 = \frac{\sum_{j=k+1}^{n} \lambda_j}{V}$$

As you can see that this is not fraction of (Refer Time: 19:36), this is a fraction of variances which are rejected.

(Refer Slide Time: 19:50)

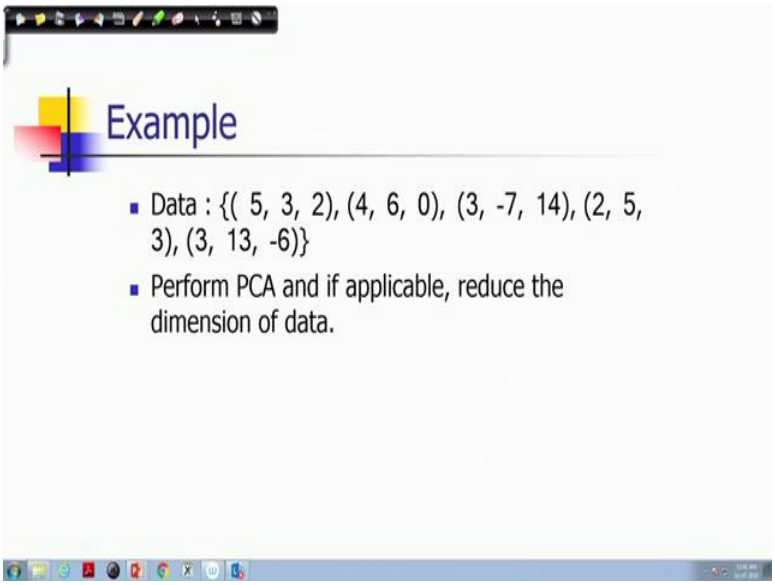


So, it is a sum of those components from variances of those components which are not accounted in your representation. So, this R^2 should be as small as possible. So, to summarize the PCA algorithm is that you have an input, it has a set of data points as we mentioned that Xj and each data point is a point in the n dimensional space. And, then the output should be a set of k eigen vectors, now k will be determined by the threshold fraction threshold that fraction of variances that we are accounting for with that particular factor.

So, the algorithm goes like this you have to compute the mean of data points, and then translate all data points to their mean. Compute covariance matrix of the set, then compute

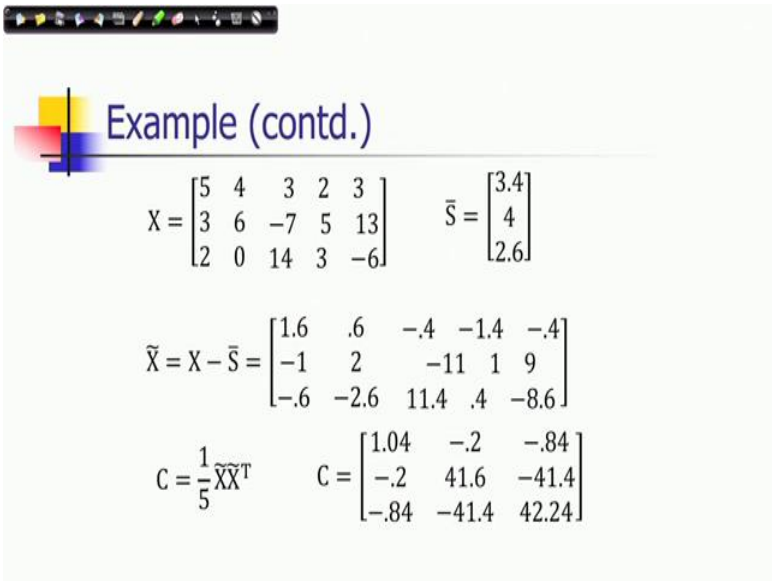
eigenvectors and eigenvalues that is an increasing order then choose k such that the fraction of variance accounted for is more than a threshold so, that threshold is also a parameter to this algorithm. Usually the typical value could be say 0.95; that means, 95 percent of the variance you are taking care of by this representation, and use those k components for representing any data point.

(Refer Slide Time: 21:05)



So, let me explain elaborate these computations using an example, consider this is a data point and we want to do PCA on this data point for reducing the dimension of the data.

(Refer Slide Time: 21:23)

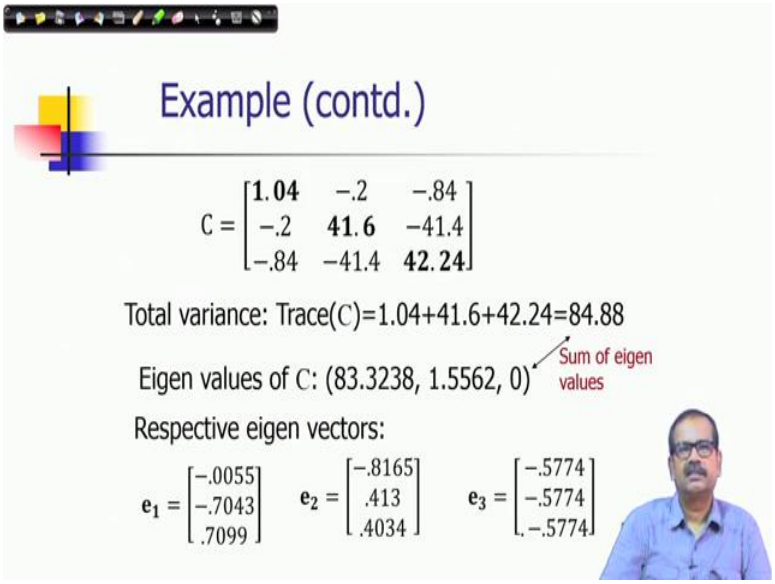


So, all these data points now they are represented as a column vector of a matrix X, there are 5 data points as you see they are all data points are in three dimensional spaces. So, each column vector is a data point so, these are data points in that three dimensional space those are the coordinates.

So, if I take the mean of those data points it is at all these three dimensional points so; that means, mean of those columns column vectors your mean is computed in this form. Now, you compute the vectors translated towards the mean. So, you simply subtract mean from the from each column vector, you will get this particular matrix X tilde, which is representing the translated vectors to around in the mean. And, then you compute this particular covariance matrix; which is $\frac{1}{5}\tilde{X}\tilde{X}^T$.

So, if I perform these matrix computations this is a covariance matrix you get. So, for principal component analysis what you need to do? You need to find out the eigenvectors and eigen values of this covariance matrices. So, since this is a 3 X 3 covariance matrix and it is a symmetric covariance matrix; you have 3 eigen values. It could be distinct, it could be non-distinct, but there were 3 eigen values and correspondingly there will be 3 eigen vectors.

(Refer Slide Time: 22:57)



In this particular example we also note the diagonal elements of this covariance matrix, actually diagonals they represent variances of components. Say first component variance

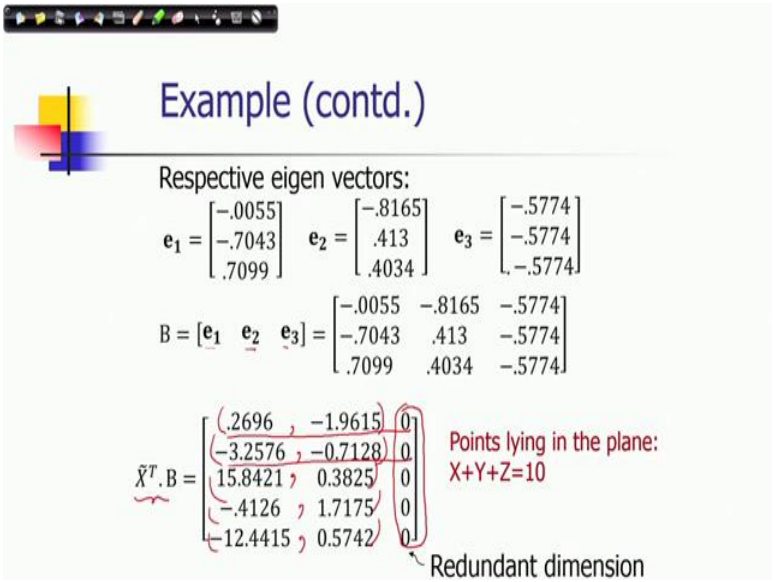
is 1.04 that is a translated, but around the mean so, variance of first company is 1.04, second component is 41.6 and third component is 42.24.

If you note that the maximum component actually maximum variance is actually in the third component in this particular data set which has been given in its original form, and also there are high correlations between the factors. Because, if you note the corresponding off diagonal terms, you will find that these terms are quite significant. So, the total variance could be, total variance is the sum of all these diagonal terms.

So, if I take the diagonal terms you can see the total variance is 84.88. And eigen values of the covariance matrix they can be computed and they are computed as; you can see that one of the eigen value becomes 0 that is the minimum, but the maximum is 83.3238 in this case which is quite high and which is capturing almost all the variance of the data, but the second eigen value is also 1.5562 which is also; it is much less than the first component, but still it has some significant component.

So, what we can do? We can represent this data because the third is 0. So, we can represent this data only using these two components itself and let us see what are the eigen. And, you should note that some of these eigen values they are the same as the total variance of the data. So, the respective eigenvectors with respect to the eigen values; that means, e1; e1 corresponds to the eigen vector 83.3238; e2 and e3 similarly they correspond to the eigenvectors 1.5562 and the other one is 0. So, we can perform the dimension reduction by considering projections alone e1 and e2 only.

(Refer Slide Time: 25:12)

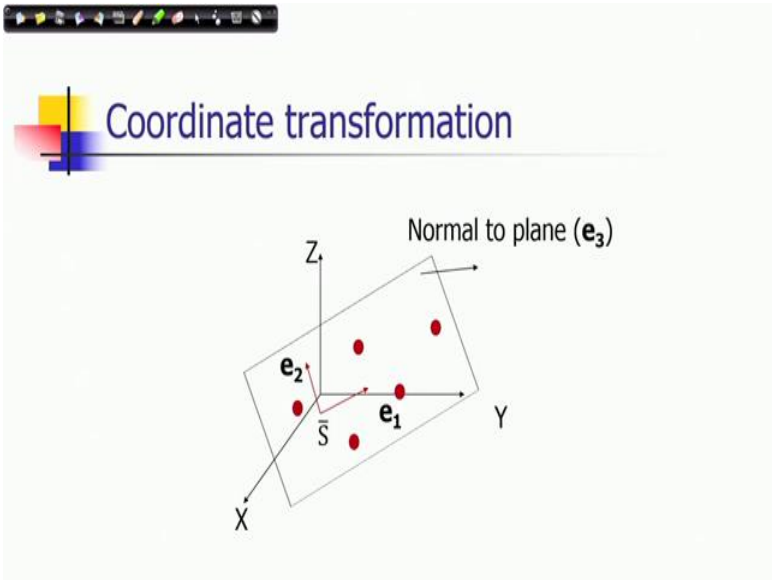


So, what we will do? We will consider as if they are the basis vectors now, and each basis vectors as the columns e 1 e 2 e 3 is a similarity there is an, it is almost it is the same as what we learned for image transform. So, you have a new basis vectors and the original data point can be you know their components with respect to this new basis vectors can be computed of course, you have to translate towards mean for principal component analysis.

So, if I consider the components of translated data points at mean of those original data points then with these computations we are computing each one of them. So, it is taking care of for each data point, it is computing the translation, it is computing the corresponding product, corresponding dot product along e 1 e 2 and e 3. So, now this is your new data point, translator and you can see that one of the component is 0 which means: now I can represent it in a two dimensional space.

So, I can represent it this is a data point, this is another data point, this is another data point like this ignoring the third dimensions. So, this is a redundant dimension. In fact, if you note the point set what I have given they are lying in the plane X+Y+Z=10. And, that is why since they are lying in a two dimensional plane; through principle component analysis what you could find out the plane of why they are lying. In fact, the third eigen value that would give you the normal to that plane.

(Refer Slide Time: 27:11)



And, that is why any projection along that direction it is 0 and e1 and e2 they are giving two directions which those are the vectors which are lying on that plane, and this is a new axis and in that plane you can once again express the coordinates of each data points. So, this is how using principal component analysis you can find out the lower dimensions on which data points lie.

So, let me stop here and we will continue this topic in the next lectures.

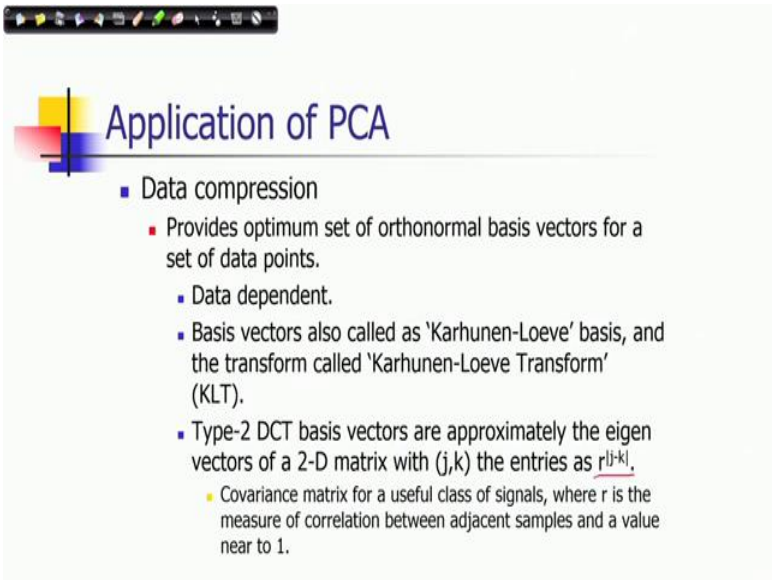
Thank you very much for listening to my talk.

Keywords: Principal component analysis, dimensionality reduction, maximizing variance.

Computer Vision
Prof. Jayanta Mukhopadhyay
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

Lecture - 52
Dimension Reduction and Sparse Representation Part - II

(Refer Slide Time: 00:22)



In the previous lecture, we discussed about the computation of principle component analysis and how we can find out the lower dimensional representations of data points, which are represented in a particular space. Now, let us consider what are different kinds of applications those are possible using PCA. Now, I will consider three major applications here.

So, one of them is a data compression. As you have already seen directly that, actually you do not require that many dimensions to represent data, that itself gives a gives an efficiency of data representation, it requires less storage for representing data. So, it provides an optimum set of orthonormal basis vectors for a set of data points. Because, we have seen in the case of image transforms, that ortho normal basis vectors they are very convenient to transform any data point into another space, and if your basis vectors are properly chosen, then you can reduce the redundancy of that representations.

So in PCA you have that advantage actually it is optimum set of orthonormal basis vectors, which will give you that kind of transformation, but it is data dependent that is one issue

here. So, for every set of data points you need to perform it is analysis, perform this analysis and then compute a new set of orthonormal basis vectors, which is not very convenient from the point of view of data compression. Because with the compressed data, then you have to also convey this information of orthonormal basis vectors which is an overview for this.

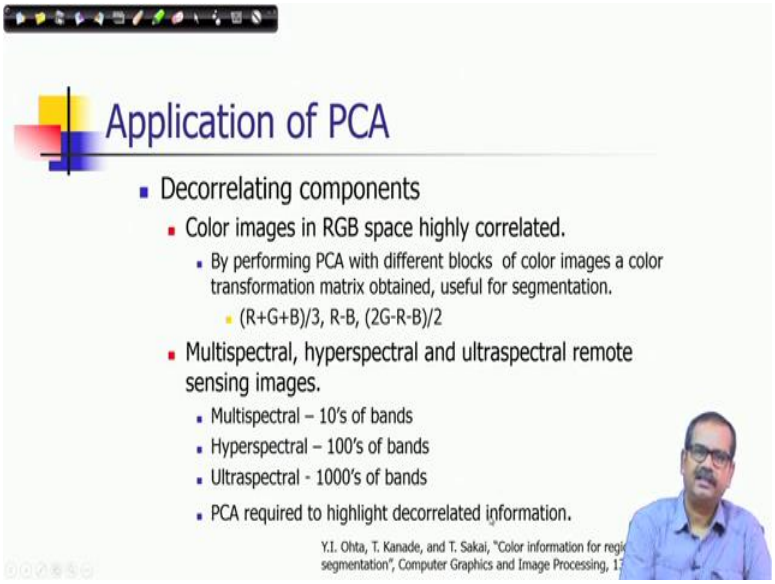
However, and these basis victors they are called ‘Karhunen-Loeve’ basis or the transform is called Karhunen-Loeve transforms which is an optimal representation of data as I mentioned. In fact, many standard basis vectors, they can be shown as eigenvectors of certain statistical representation of signals or images.

For example, type-2 DCT basis vectors are shown as it is approximately the eigenvictors of 2-D matrix with j, kth entries as $r^{|j-k|}$ as it is shown here. So, these are it represents the correlation between adjacent samples say j and k. So, if j and k it is deviates too much from that location it, then it is not it adjutancy is less and then the correlation should be less.

So, because r is a value which is less than 1, so as the j - k this magnitude increases it will be reducing this value will be decreasing. So, you have less correlations, but for high correlations it, so for adjacent samples it is expected they should be very highly correlated. Now, with this kind of data, it has been shown that the basis vectors are; that means, eigenvectors are similar to the almost similar to the type-2 DCT basis vectors. And, that is why type-2 o DCT is so, efficient for representing a large class of signals and images.

So, as I mentioned covariance matrix for a very useful class of signals, where r is major or of correlation between adjacent samples and it is the value with me at to 1. So, it is trying to represent the natural images and natural signals with these kinds of statistics it is trying to model that part.

(Refer Slide Time: 04:02)



The other advantage of PCA or other applicants which is used also different applications, that it de-correlates the components. We have already seen in the example that covariance matrix they show that there is high correlations between different components for the original data. But, after performing PCA actually those correlations they are those correlations are largely reduced. So, it will be almost 0 in that case ideally it would be 0.

So, that is how it de-correlates the components. So, one of the applications of this kind of using this property is that you can find out a new color space, that when color images they are represented in RGB color space they are highly correlated. So, there is a work I have given this reference here, it is the work done by Ohta Kanade and Sakai. So, what they did, that they took a large number of color images and, they performed PCA using the different blocks of the color images. And, then found out eigenvalues and eigenvectors and those eigenvectors that is the data that gives you the transformation new transformation space.

So, in fact, they found that if I transform the color component using those eigenvectors in this way, that (R+G+B)/3, R - B is a second component and (2G-R-B)/2, there is a third component. Now, these are the principle components from their principal component analysis.

And, as you can see this is nothing, but you are performing a color transformations where the fast one is indicating the intensity value and the other two they are the chromatic

components. So, this can be, this has been obtained through this PCA itself. The major applications of PCA it is there when you have more number of components in images.

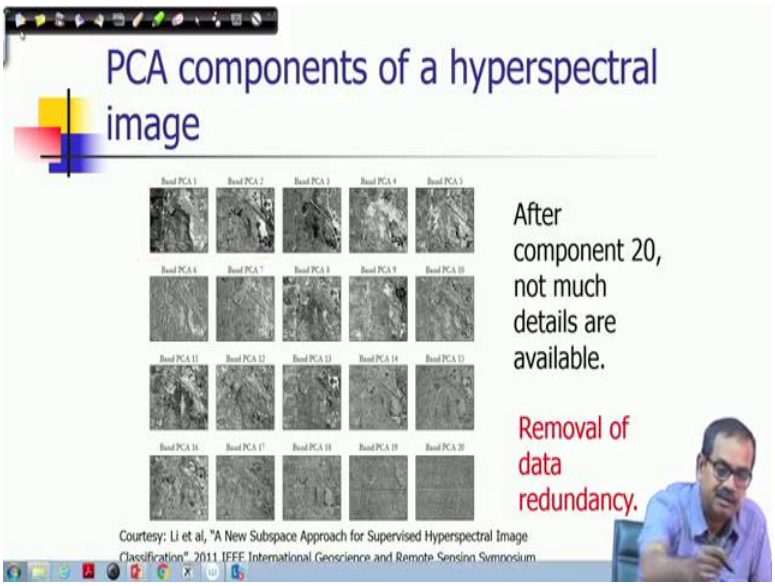
So, color images they have only three components, but if I consider remote sensing images, these components could be very large there are different kinds of remote sensing images depending upon the bands they are using, depending upon the electromagnetic wave length bands they are using. So, multispectral, hyperspectral, ultra spectral, remote sensing images. And, there could be many bands, like multi-spectral it could be 10's of bands, hyperspectral it could be 100's of bands, ultra-spectral it could be 1000's of bands.

We have so many different bands so many numbers of bands and how to get an efficient data representation using those bands. So, what we can do? So, which means that at every pixel, it has dimension; data dimension is say if it is band number is N, data I mentioned is N.

So there are, these kinds of dimensional reduction becomes very useful we can find out only those components after deduction. We can reduce those components which are de-correlated and use those components to make the information analysis to analyze processing permission or to correlate with different ground truth or different ground level information.

So, PCA is required to highlight the correlated information.

(Refer Slide Time: 07:41)

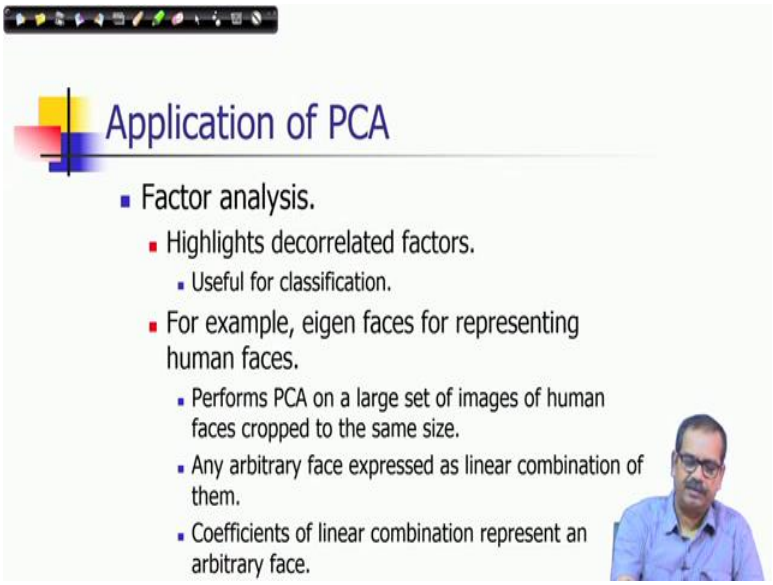


This is one example where I am showing that PCA component of a hyperspectral image and you can see that it starts from this, this is the PCA, which will corresponds to the maximum variance, maximum eigenvalue, you have lots of information there, a lot of details there and this is a minimum eigenvalue in this case it is actually 12th eigenvalue.

Now, if the number of band is in this case it is not specified, I suppose the number of bind is 20 in this case. So, this is a minimum one. In fact, there is no such information also almost no details are visible here. So, that is the advantage of PCA, you can prioritize or you can give references to those bands which are having more informations.

And, say this is a PCA 1, this is band PCA 2 band, second band, third band as you progressively go over the PCA components from say top to bottom and left to right by making these kind of scan. You will find finally, the details are slowly dying and it gets almost like a smooth region. And, so, you can use this components to find the, to analyze the information where if it is representing those information which right here. So this is what after component when (Refer Time: 09:22) much details are available and it removes that data redundancy in this representation.

(Refer Slide Time: 09:29)

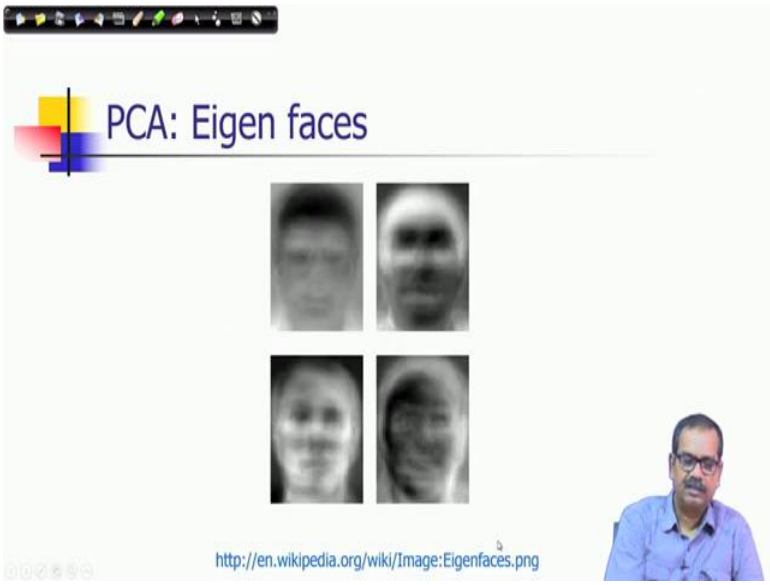


The third application of PCA could be factor analysis and it highlights this decorrelated factors. So, you can find out the factors related to that even the color component what I have shown, it is trying to find out those factors, intensity factors, coma factors and this

factor analysis this is very much useful for classification. For example, you consider eigenvalues for representing human, human faces.

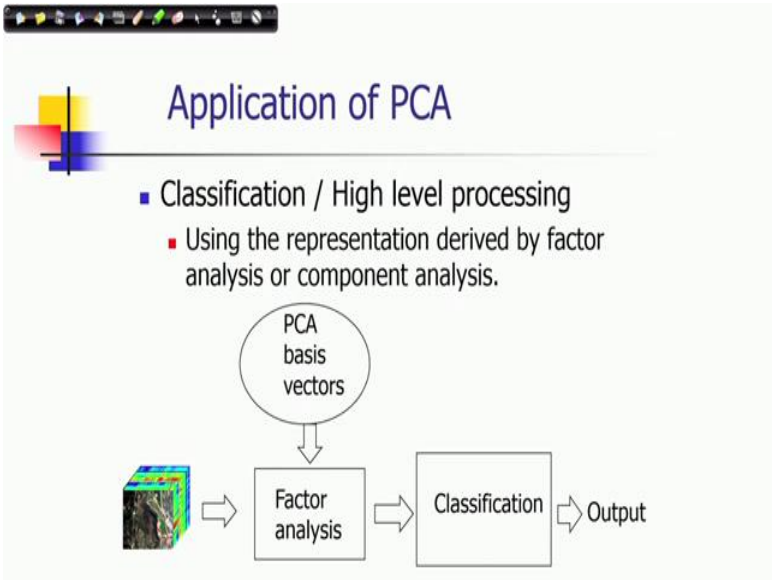
So, what we are doing here, we consider a large set of images of human faces crop to the same size and follow certain rules like it is not only just simple cropping, you are trying to maintain the different parts of the human face from the similar distances from the top. And, then you perform PCA and any arbitrary face you can express as a linear combination of those eigenfaces; that means, if you perform PCA you get eigenvectors those are called eigenfaces here. So, coefficients of linear combination it represent an arbitrary face.

(Refer Slide Time: 10:49)



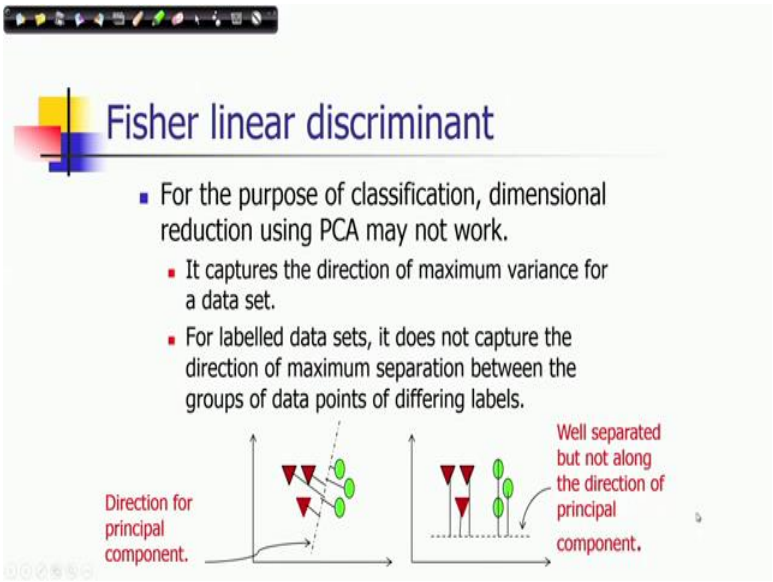
So, this is one example that it is showing four eigenfaces. So, if I consider any arbitrary face at least I can represent by four-dimensional vector using this PCA, using these factors. And, then subsequently we can use it for classification.

(Refer Slide Time: 11:08)



So, this is the another applications, that we use those factor analysis, use those representation or any classification or high level processing like, you can this is a pipeline. So, you can perform factor analysis and then those representation can we use for classification.

(Refer Slide Time: 11:31)



So, I will be now discussing another type of dimension reduction. In fact, the objective here is more for the purpose of classification. In the previous slide itself I have shown how PCAs could be used for factor analysis and those factors could be used for classification.

But, it is not necessarily that those representation of those factors are efficient for classifications particularly if you are considering linear discriminate functions.

So, we will be considering those kinds of dimensional reductions, where this linear discrimination becomes simple for in fact, this is a simple discrimination it is you are reducing it to just one dimensional component. So, the objective of this linear discriminates and which is known as special linear discriminate by the inventor Ronald Fisher is a very famous statistician and it captures the direction of maximum variance of a dataset.

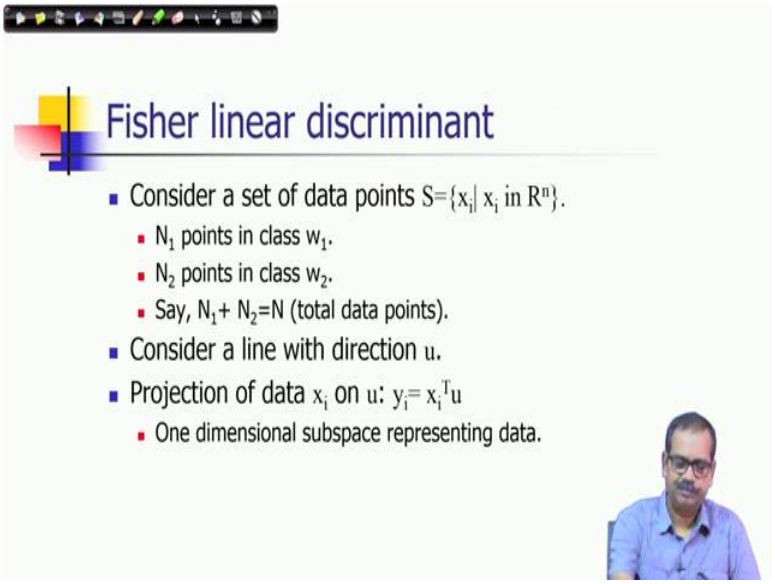
So, for label dataset it does not capture the direction of maximum separation between the groups of data points of different levels. So, here the point is that PCA, it captures the direction of maximum variance for the data set not the vicious linear discriminate. And, but if I consider the dataset is labeled then it may not capture the maximum separation. So, I will be showing you with I will be explaining it with respect to this diagram consider there are two groups of data points, these are the labeled data points; one is shown by the triangles, another other groups are shown by ellipses by different colors.

Now, the direction of maximum variance is a dotted line, that could be the direction, along that directions all the projections of these data points, they will have maximum variance. But, if I take the projections as you can see here that within these intervals the projected data points there they are intermingled. So, they are not well separated. So, they cannot be segregated using simple interval rules, they cannot be discriminated by that rules.

But whereas, if I consider another direction say this is a new direction another directions and if I take the projections. Now, you can see that all the projected points for this group they are lying in this within this interval and all the projected points from for these group they are lying within this interval along this direction of projection along that particular direction. So, they are well separated.

And, as it; so, this shows direction of the principal component is not really providing you the good separations between data points. So, as we mentioned it is well separated, but not along the direction of principal component.

(Refer Slide Time: 14:51)

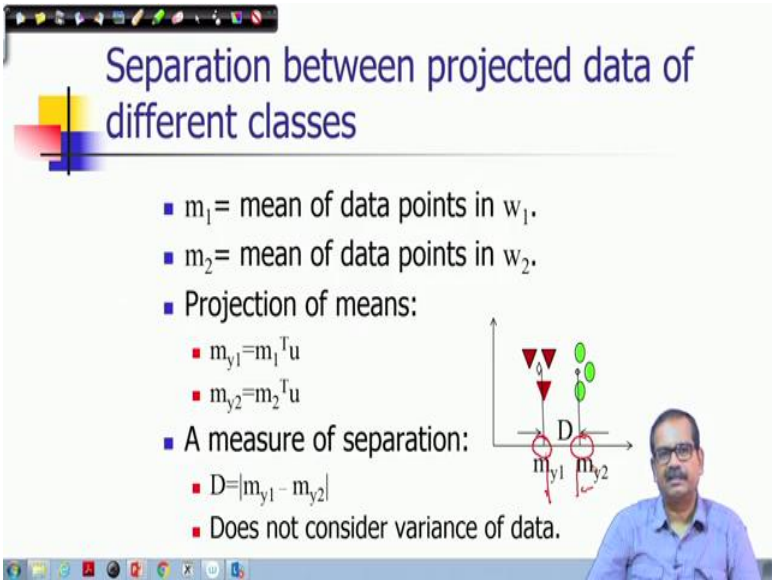


So, let me explain that what is the competition problem that involves in this particular analysis; Fishers linear discriminant analysis.

So, consider a set of data points as we have considered in the previous cases also. And, then out of those there are N1 data points which are in class w1. And, there are N2 data points which are in class w2. So, naturally we consider that N1+N2=N which is a total number of data points. And, now you consider a line with direction u, because our objective is to get a direction where this separation could be maximum. So, we should consider projection of data xi on u.

So, the projection can be operation can be expressed as a dot product of two vector xi and u or in the matrix representation, it is $xi^T.u$ matrix multiplication of xi^T and u. And that would give you the projection of data point yi, it is nothing, but a 1-dimensional sub space, which represents this data so, all the points around that lying on that line.

(Refer Slide Time: 16:18)

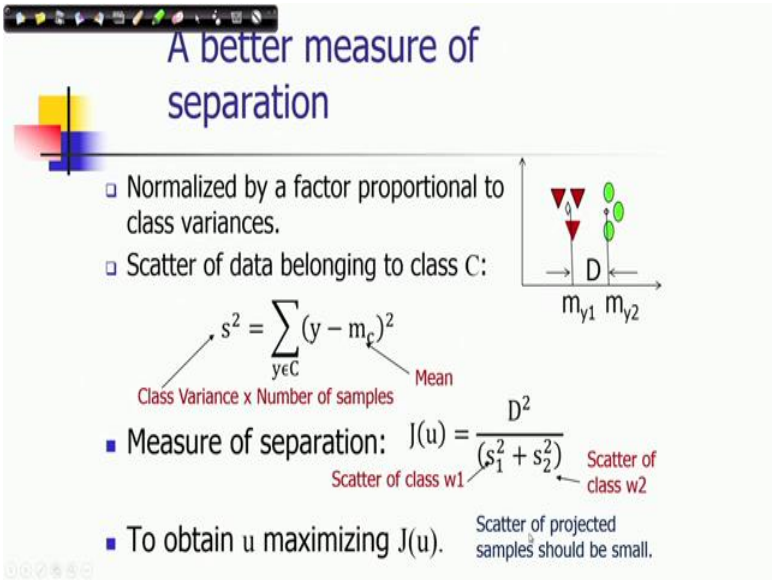


Now, we are trying to measure, what is how the projected data is separated? Because with that major then we can formulate a problem for maximizing that measure to get it well separated data. So, in this case let us consider mean of data points in class w1 is m1, and mean of data points in class w2 is m2. And, projection of this means along this direction u can be computed as projection of mean vector along u that $m_1^T u$ and $m_2^T u$

So, figuratively we can show here that m_{y1} and m_{y2} and one of measure of separation could be the separation of these two means; that means, how far there, we want this measure should be no large, this value should be large per well separated two groups.

So, if you consider the difference absolute difference between these two values, that would give you a separation measure, but the problem here is that it is not capturing the variance of data. So, some data could be very well spreaded, very largely spread it, in an area and some could be very closely groups, closely spaced and after projection. So, ideally what we would like to have, ideally you should have the data points they should be closely spaced around mean. And, they should be largely separated. Then only those kinds of measures I mean those are the desirable situations desirable cases for this projection, but that is not captured by simply by the value D.

(Refer Slide Time: 18:13)



So, what we do in that case we normalized this D by a factor proportional to class variance. So, in fact, we call it this factor scattered. So, scatter of data belonging to class C is defined in this way it is on the projected space.

$$s^2 = \sum_{y \in C} (y - m_c)^2$$

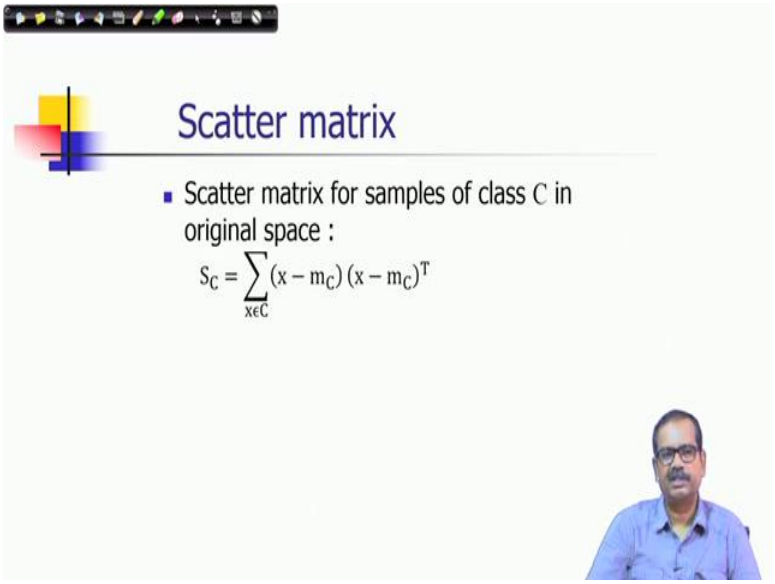
So, it is nothing, but as you can see, it is a sum of mean square of mean deviations from the mean from the mean of the data points. And, it is a proportional factor with variance, because it is nothing, but class variance product of class variance and number of samples. And, as I mentioned the m c here represents of the mean of class C and small s square is a scatter.

So, the measure of separation a good measure of separations could be that

$$J(u) = \frac{D^2}{(S_1^2 + S_2^2)}$$

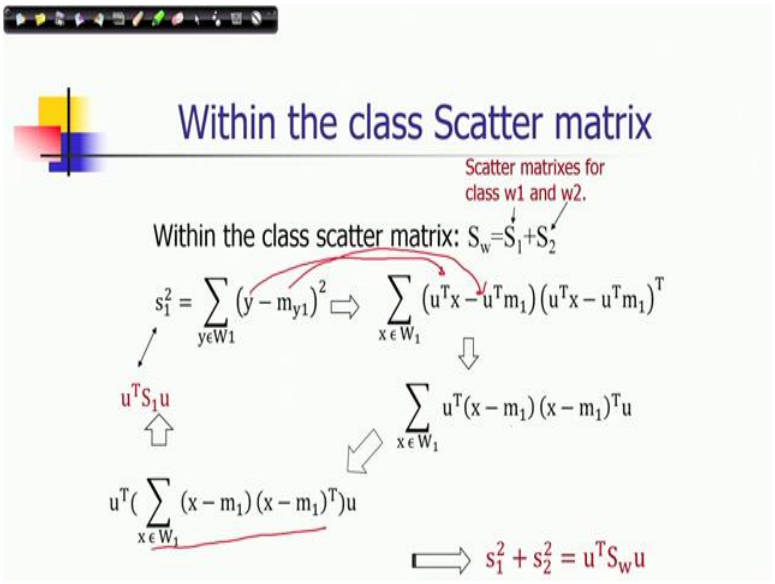
And, our objective is to maximize this value J(u) with respect to get a direction and so, scatter of projected samples should be small.

(Refer Slide Time: 19:44)



Let us define also scatter matrix of the original data. So, scatter matrix of sample of class C in the original space that is also defined in the same way. So as you can see it is the multidimensional space. So, earlier we have only one-dimensional space sub space for projections. So, we have used standard definitions of variances kind of definitions; that means, square deviations, but now you have to use the corresponding outer product of this matrix to denote a scatter matrix. So, this is represented as, as you can see

(Refer Slide Time: 20:33)



So, within the class scatter matrix, that is defined as scatter matrix for S1 and S2; sum of S1 and S2 that is within the class scatter matrix. So, each one for S 1 you have to consider once again that definition; that means, suppose for S1, we had m1 as the mean. So the vector $(x - m_1)(x - m_1)^T$ and sum of if I consider there are ith samples. So, these ith component and if there are N1 samples in S1.

So, we can show how it is related with this within the class scatter matrix. So, this can be written as in this form. So, you can see that this is what is the projected sample of x along the direction u. So, we can write $u^T x$; similarly, this is the definition of this is a projected mean of the samples along y along direction u. So, it is $u^T m_1$. So, this itself can be written

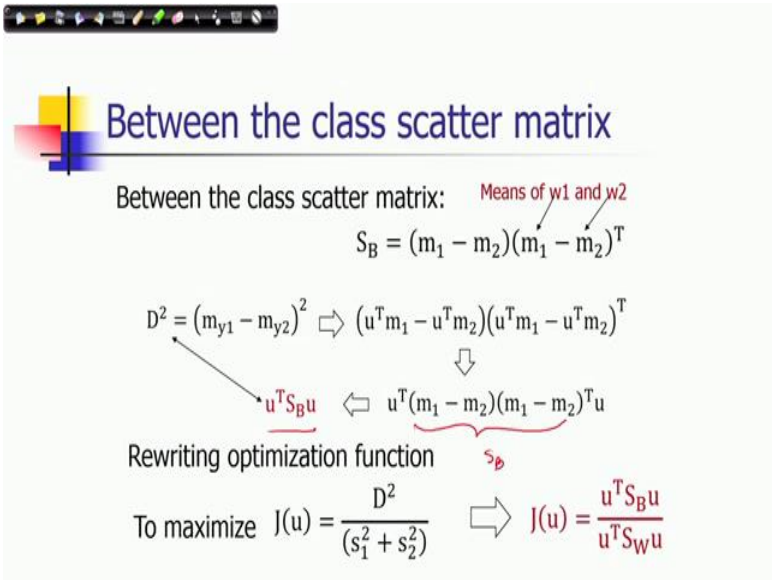
$$s_1^2 = \sum_{x \in W_1} u^T (x - m_1)(x - m_1)^T u$$

So, we can write in this way you can simplify this algebraic form and you can write in this form. You can note here actually since it is one-dimension. So, whether I put the transpose here or here does not matter. So, we have taken the convenient form for the sake of derivations for the sake of the final derivations. So, we can see that from here you can take out u transpose u outside and this will give you the scatter of the original sample for class W1.

And, this is the reason y^T here instead of taking u transpose a minus this could be as transposed and you could have taken this also. On the other hand we may and (Refer Time: 23:17) this is also correct. So but, the advantage of this notation is that, then we can bring the scattered matrix nicely within this u^T and u in this form, we can get it in this form.

And so, from here we can see that a small s_1^2 is related with the scatter matrix of that class by this explanation. So, s_1^2 is nothing, but $u^T s_1 u$. Similarly, s_1^2 would be $u^T s_2 u$ and if I add them finally, the both $S_1^2 + S_2$ square can be written as $u^T S_w u$, where Sw is the within the class scatter matrix as we have defined.

(Refer Slide Time: 24:08)



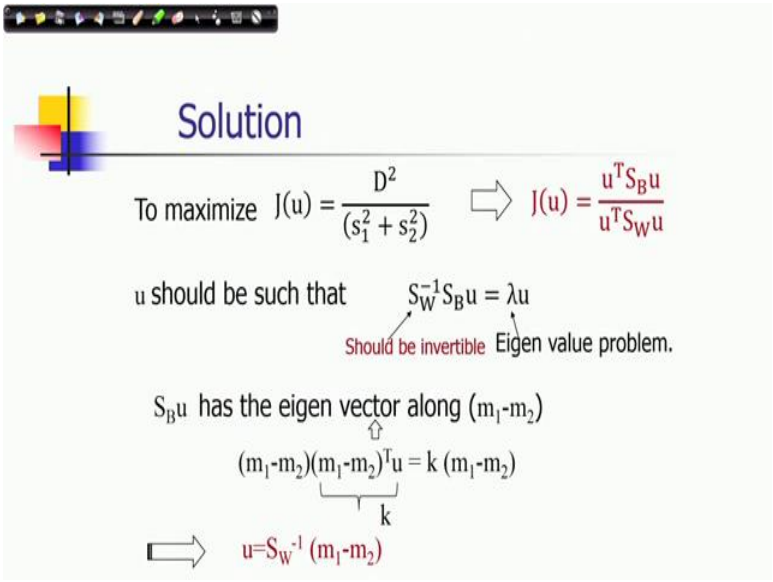
Between the class scatter Matrix; that is another definition. Now, this is the scatter matrix formed by the means of those classes. So, this is how the definition is there. So, $S_B = (m_1-m_2)(m_1-m_2)^T$, it is the difference between two mean vectors. So, difference vector of means that is defining this scatter matrix. And, here also we can show that the separation is related with this.

So the D^2 which is nothing, but the square deviation of two projected means that could be written in this form. Once again conveniently we have used the outer product from 2 to outer product matrix representation for showing the square of a one- dimensional term and this could be represented as

$$J(u) = \frac{u^T S_B u}{u^T S_W u}$$

Now, this part is nothing, but this is what is S_B and so, you can write it in this way and that is related with D^2. So, finally, we can rewrite this optimization function as it is to maximize these factor J(u) which is $\frac{D^2}{(S_1^2+S_2^2)}$ this could be re written as $\frac{u^T S_B u}{u^T S_W u}$.

(Refer Slide Time: 25:51)



So, this is the problem of optimization, that you have to maximize it, once again u should be unit victor that is a constraint.

So, u should be such that it can be shown that

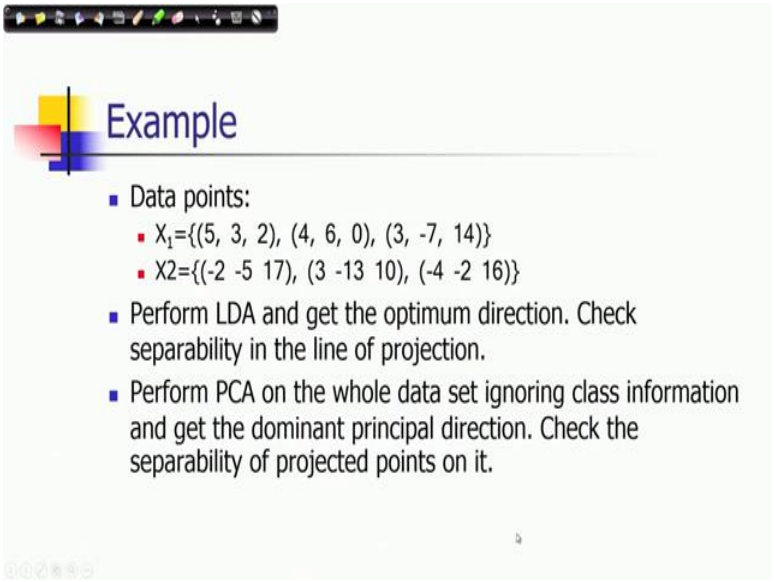
$$S_W^{-1} S_B u = \lambda u, S_W \; is \; invertible$$

So, I am not giving you the divisions I am providing you the final solutions that u is the eigenvector of these particular matrix. It is like an eigenvector solution $S_W^{-1} S_B$ and if we compute the eigenvector there. And, since now you want maximization which means you have to consider the maximum eigenvector corresponds to the maximum eigenvalue.

So, and one of the thing is that interestingly that $S_B u$ it has eigenvector along already (m1-m2), this can be shown easily, if I consider this particular expansion of S_B. So, this is nothing, but S_B. So, this part as you can see that this is the dot product of the difference vector with respect to u and we should be some scalar value. So, this is some scalar value k.

So, finally, this expression is nothing, but it is a vector this is some scalar term this is nothing, but a vector m1 - m2. So, the eigenvector of SB is also m1-m2. So, in that case the solution of $u = S_W^{-1}(m_1 - m_2)$, that is what is your solution and, in this way you can get the direction where you can get the maximum separation of projected samples; this is how the Fisher's linear discriminant works.

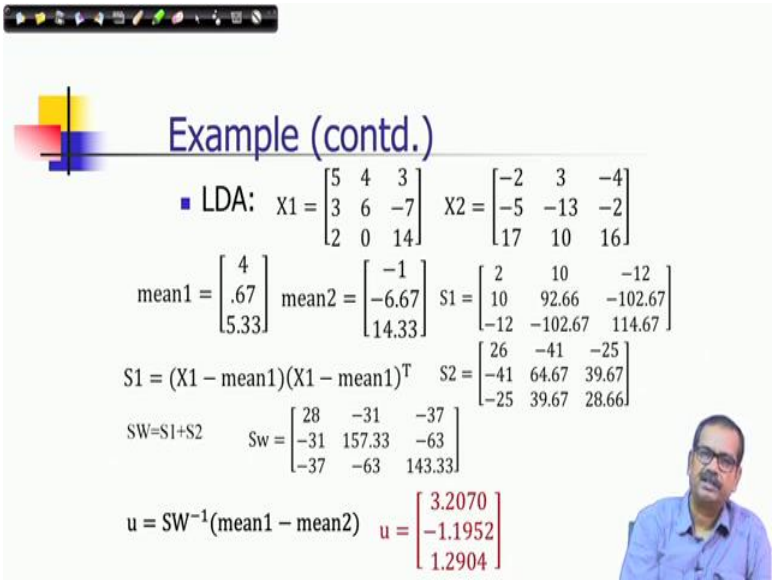
(Refer Slide Time: 27:47)



So, let me explain it with respect to an example consider a set of data points and now we are considering labeled set of data points. So, we have I show given you 2 such sets X1 and X2 and we perform linear. So, we have to perform linear discriminant analysis or to get the optimum direction or FLD actually Fisher's linear discriminant analysis to get the optimum direction.

And, we should check also the separability in the line of projections. As an alternative also we will perform PCA on the whole data set, because PCA does not consider class information, it considers the whole data set and get the dominant principle direction and check the separability of the projected points on it.

(Refer Slide Time: 28:38)

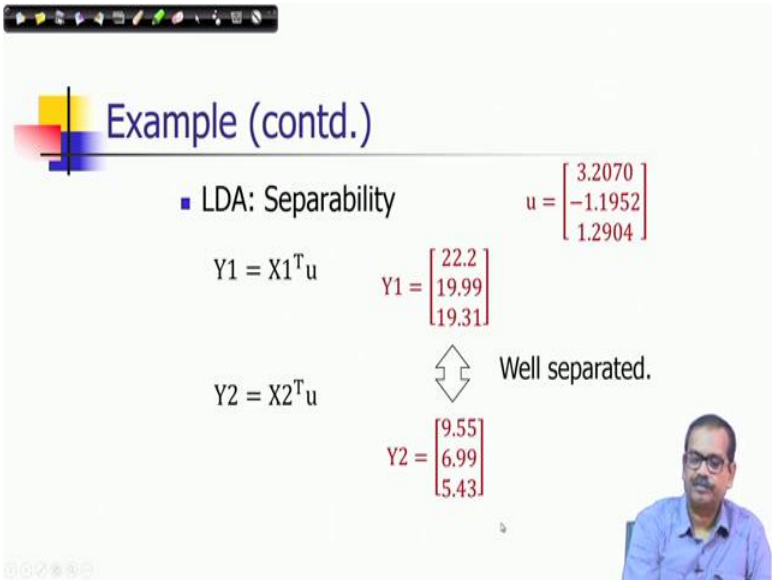


So, for a linear discriminant analysis or performing here Fisher linear discriminant, we consider this representation of data into two matrixes X1, 1 group of data all column vectors are data points X2, other column vector are here also column vector data points per group 2.

So, mean of X1 is given this that is what is m1 and mean 2 is given this, that is what is m 2 in our notations we have discussed earlier and this is what is your scatter matrix for the X 1. So, S1 is a scatter matrix for S1; similarly, you computes scatter matrix for X2 S2 and we within class scatter matrix can be represented. Now, this is a definition of S1, which we already we have discussed. So, within class scatter matrix is S1+S2, which is given this. And, now the solution you have to take the S_W^{-1} and then multiply that with m1-m2 that would give you the direction.

So this is what is your u, S_W^{-1}(m1-m2) and it could be found that u value is given this, it could be unit vector also it could be any vector, because it is just projection. So, magnitude of the vector does not.

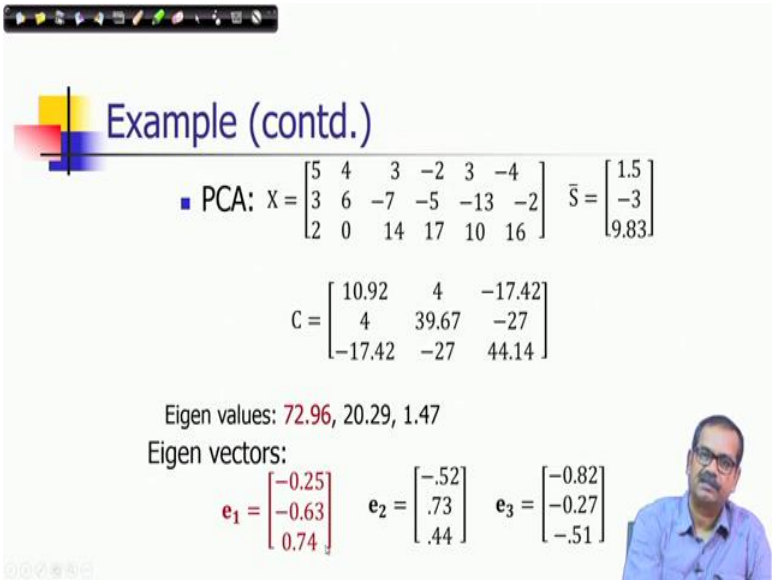
(Refer Slide Time: 30:08)



So, the separability if you would like to study you take $Y_1 = X_1^T u$. So, you are taking the projections of all data points in X1 with respect to u. So, you can perform all these operations simultaneously using these matrix multiplications. Similarly, you consider projections of all data points of X2 with respect u.

So, you will get them as column vectors there are three data points here. So, these are the projections for Y1, these are the projections for Y2. And, you can see that their intervals are well separated, because the range of Y1 is 19.31 to 22.2 whereas, range of Y2 is 5.43 to 9.55 and they are well separated.

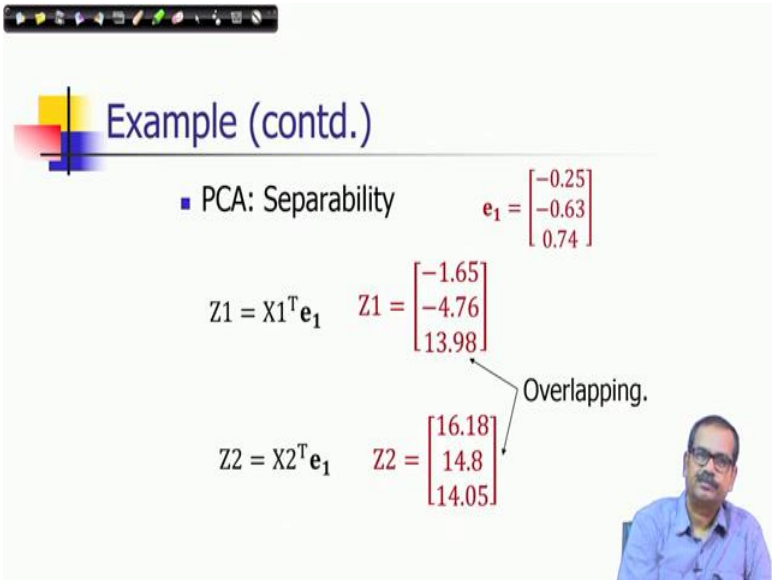
(Refer Slide Time: 30:55)



Whereas, if we take principle, if you perform principle component analysis on the whole data set which is represented here by the matrix X all column vectors are data points and you have considered union of data points of X1 and X2. So, part principle component analysis as we did earlier, first we compute the mean vector S bar, then you compute the covariance matrix, the variance of data.

So, sum of diagonal elements that would give you the total variance of the data. And, if I perform the Eigen; compute the eigenvalues and eigenvectors in order of there, values I am showing here. So, the maximum eigenvalue is 72.96 and eigenvectors corresponding eigenvectors are this e1, e2, e3 and we are interested on finding out the directions of maximum eigenvalue or maximum variance. So, this is your principle component direction of the principle component is given by e1.

(Refer Slide Time: 31:58)



So, if I take that direction and check the separability of projected samples so, you perform the same projections on that direction and these are the values we will get. We will see that at least there is one sample this is the (Refer Time: 32:14) example. So, I mean it approximately it is good well separated, but there is at least compared to the previous one, one sample is here 13.98, which lies in the interval or very close to the interval of the data points of Z 2.

So, the separation is not that much, it is not really (Refer Time: 32:38) overlapping in this sense here, but it is very close to that integral. So, that is it shows that the utility of fishers discriminate analysis. So, with that let me stop here and I will continue this topic in the next lectures.

Thank you very much for listening to my talk.

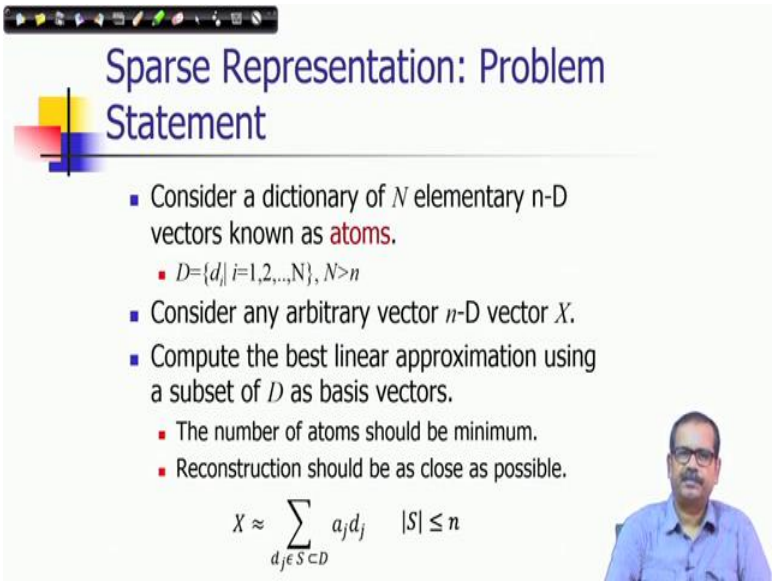
Keywords: Fischer linear discriminant, between class variance, scatter matrix, eigen faces.

Computer Vision
Prof. Jayanta Mukhopadhyay
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

Lecture - 53
Dimensions Reduction and Sparse Representation Part – III

In previous lectures we discussed about dimension reduction of data points.

(Refer Slide Time: 00:23)



Now, I will be discussing another issues related to data representation which is called sparse representation. So, let us understand the problem statement involved in this particular issue. Consider we have a dictionary of N elementary n D vectors we call it dictionary it is something like set of basis vectors.

But in the set of basis vectors we would like to have only those basis vectors which will be sufficient to represent data. But here we consider a redundant set of basis vectors; that means, we have more number of basis vectors and if you would be only required to represent a data.

And the term what we will be using instead of set of basis vectors we will be calling this particular collection as a dictionary. As if you have lot of redundant vectors and conveniently you would like to use them for representing your data or signal.

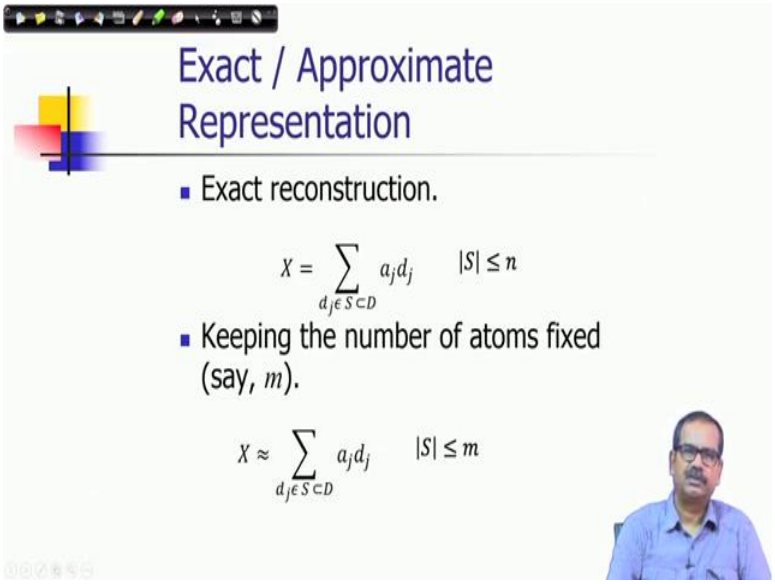
Now, the elements of this dictionary they are called atoms that is a terminology we will be using and you consider any arbitrary vector n dimensional vector. So, the problem here you have to compute the best linear approximation using a subset of D as basis vectors. As I was mentioning that D is an over complete representation of basis vectors which means there are many redundant vectors and it is not required all the vectors should be used for representing data.

So, the linear combination of the subset of D can be expressed mathematically in this form as you can see. That X is your input vector it should be very close to a linear combination which is given as a multiplication of each dictionary elements with the scalar constraint aj and some of them. And dj is are the elements of that set which has been considered for representing X and which has to be subset of this dictionary.

And in an n dimensional space it is sufficient if you have n independent basis vector, n linearly independent basis vectors and then it is sufficient you can use into represent any arbitrary vector X. So, here you have an over complete representations and you deserve that this number should be less and that is the sparsity we are considering.

That you required a few basis vectors only from D which should be sufficient to represent X we do not require N basis vector. So, it should be much smaller to n and that is what we are trying to achieve using these kinds of representation, so the idea is that this number of atom should be minimum. So, for the particular problem statement you would like to have that S subset whose cardinality should be minimum and also reconstruction should be as close as possible. So, it is an approximation.

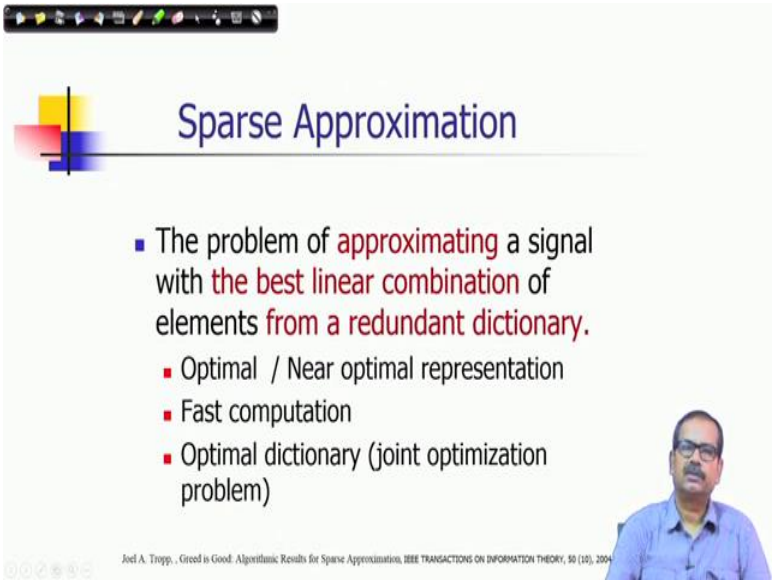
(Refer Slide Time: 03:34)



So, to make it more precise in that case how close it is first thing it could be exactly construction. So, you find out the minimums which will give you the minimum cardinality of S, a minimum number of atoms which will give you the exact reconstruction. So, this could be a problem and it is a very precise problem statement.

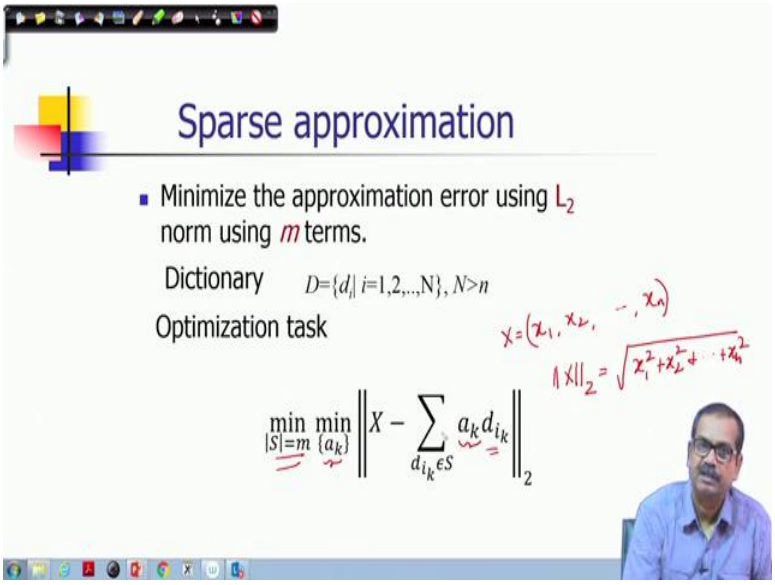
Or what you can do we can keep the number of atoms fixed say what is the best representation; that means, how close that should be the error between the representation approximation of X and X should be the minimum original value, so it should be very close to that.

(Refer Slide Time: 04:11)



So, to summarize this problem it is the problem of approximating a signal with the best linear combination of elements from a redundant dictionary. It should give some optimal or near optimal representation, the competition should be fast. These are the desirable properties and the dictionary also should be optimal and sometimes there are joint optimization problem that you would like to have also a dictionary of with an optimal size, so that gives also the optimal sparse representation of data.

(Refer Slide Time: 04:46)



So, to elaborate this optimization task we can consider this mathematical formulation that we have to minimize the approximation error using L2 norm. You know L2 norm is basically it is the Euclidean distance or in the n dimensional space and this it is we are using this and also using m terms. So, you have a dictionary D which is a set of d i's each one is an atom and this considers L 2 norm. So, just to show you what means by L2 norm, suppose you have a vector is represented as say x1, x2 to xn.

So, L2 norm is

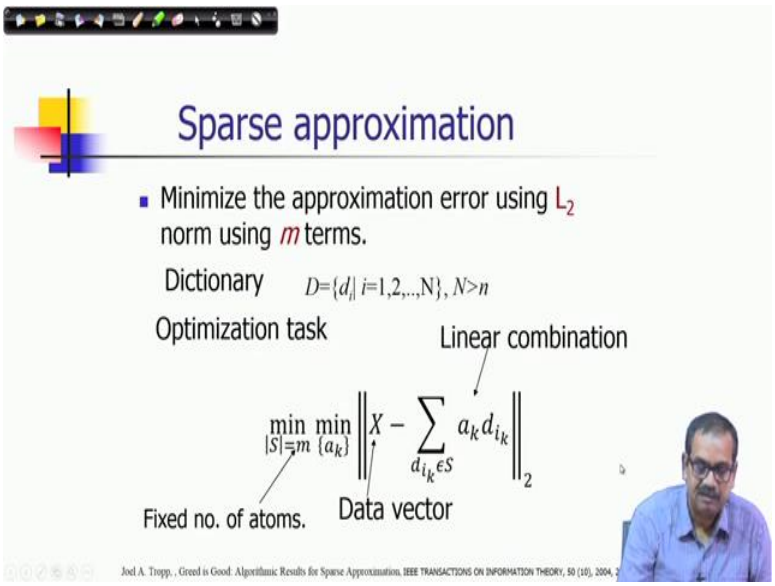
$$L_2 = \sqrt{(x_1^2 + x_2^2 + .. + x_n^2)}$$

So, you consider this is an error vector

$$\min \min | \left| X - \sum_{d_{lk} \in S} a_k d_{ik} \right| |_2$$

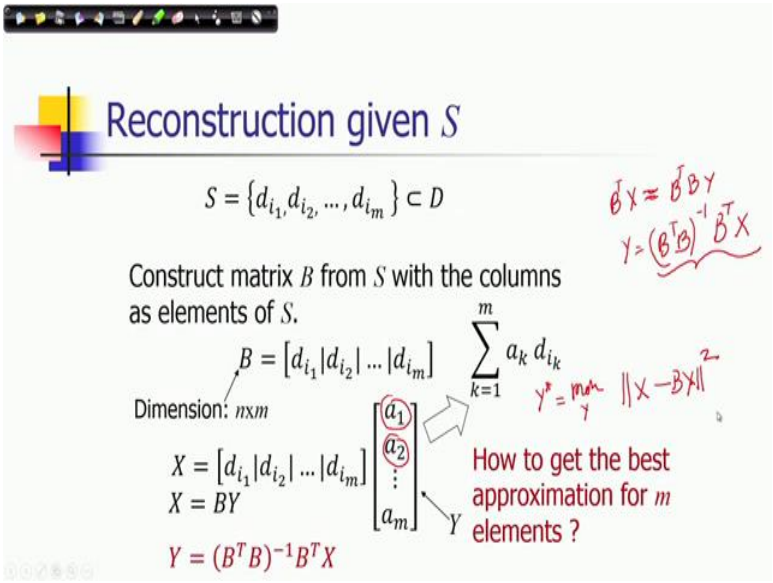
So, the problem here is that you should get that set of coefficients and the total number of atom should be exactly equal to m, so that you get the minimum error minimum L2 norm of these values.

(Refer Slide Time: 06:24)



So, these are these explaining the corresponding conference of this expression. This is the data vector and this is the linear combination and it gives you the fixed number of atoms that constraint.

(Refer Slide Time: 06:38)



Suppose I have given you a set of atoms which is a subset of the dictionary D. Now, how can I get the best reconstruction we in terms of linear combination of these atoms for in a arbitrary X, for in a arbitrary vector X. So, we can see that we can probably it easily the least square error automation problem. We construct a matrix B, so once again be relates to that setup basis vectors and each element of S is considered to be a vector this is vector.

So, this B is found in this weight, so each column vector of B is a corresponding atom of S. So, the dimension of B should be n X m as that dimension of each vector each base is for each element of d is n. So, each column victor is of n that is the dimension, so number of rows would be n and since there are m elements m atoms, so we will have m columns.

So, the dimension is n X m and if you consider the linear combination of these atoms which can be conveniently represented as in this form. So, here a1, a2 these are all scalar quantities these are all scalar. And you are multiplying $a_1 d_{i1} + a_2 d_{i2} + .. + a_k d_{ik}$ like this you proceed and you can represent in this form that is well you get the linear combination.

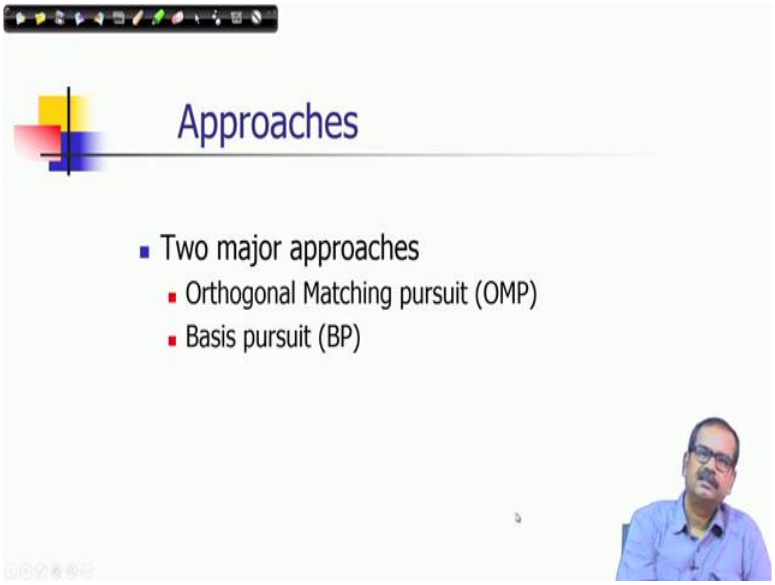
So, this linear combination can be represented and this is this set of coefficients or this column victor let me consider. Let me denote as Y and this linear combination can be

represented as the multiplication of B and Y, B is the set of basis vectors as I mentioned from the dictionary where each basis vector is the column of B. Or dictionary elements of is the column of B and Y is the corresponding set of coefficients, early stock coefficients with respect to each atom, so Y is the representation of X given this dictionary B.

So, Y is the transform of X and this is this should be sparse we would like to have Y as sparse as possible, so how to get the best approximations for m elements. As you can see this is nothing but you can convert it to a list squared error estimation because, what you can do you can minimize here B is given X is given what you have to find out? You have to find out Y.

So, minimize this norm X minus B Y squared and that Y will give you the optimal representation of this particular element representation of X with respect to these constraints. So, we know this solution a number of times you know we have we did it, because here once again just let me put it I would like to get the close approximation in this form. So, what we can do? We can multiply B transpose and then Y can be written as B transpose B inverse B transpose X. So, this is the representation this is the solution of this particular problem, so this is how you can get Y out of this process.

(Refer Slide Time: 10:56)

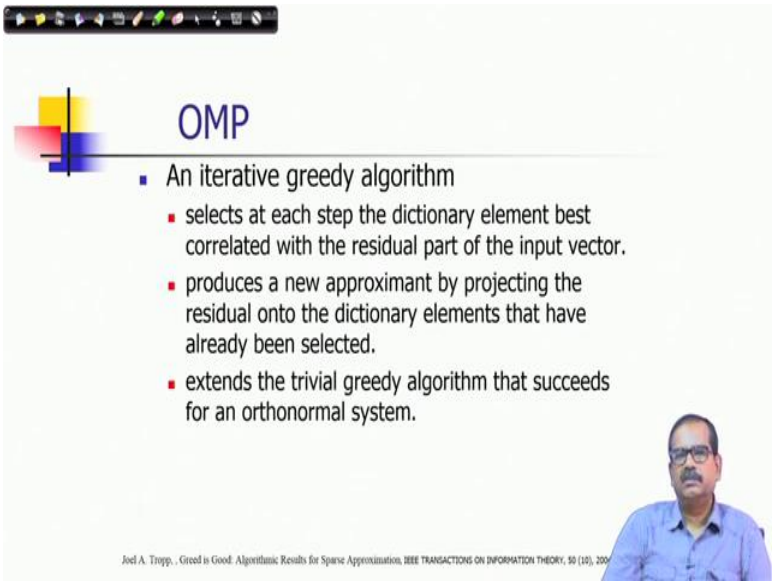


So, the problem here is that if I give you is then use the least square error estimation method to get a representation Y for particular X. But which subset you should consider, which list to m atoms you should consider from the dictionary that is what the basic problem is.

If I can if I know that this set of atoms will give me the minimum reconstruction, then our sparse representation I can easily get the solution.

So, there are various approaches to select those atoms which will derive this kind of sparse representation. Now, these approaches are called pursuit approaches, so two major approaches; one is orthogonal matching pursuit, another one is basis pursuit.

(Refer Slide Time: 11:50)



In orthogonal matching pursuit or in short we call it OMP here the algorithm is in iterative greedy algorithm, what it does it selects at each step the dictionary element which is based correlated with the residual part of the input vector. We will see what it means; that means, what it is doing it is successively refining the refining the representations it is like successive approximations.

You take the original vector X or input vector X and find out the basis vector where the component is maximum; that means, that vector is representing the maximum part of the signal. So, you choose it in your dictionary element and that is your representation itself, so that vector and the component.

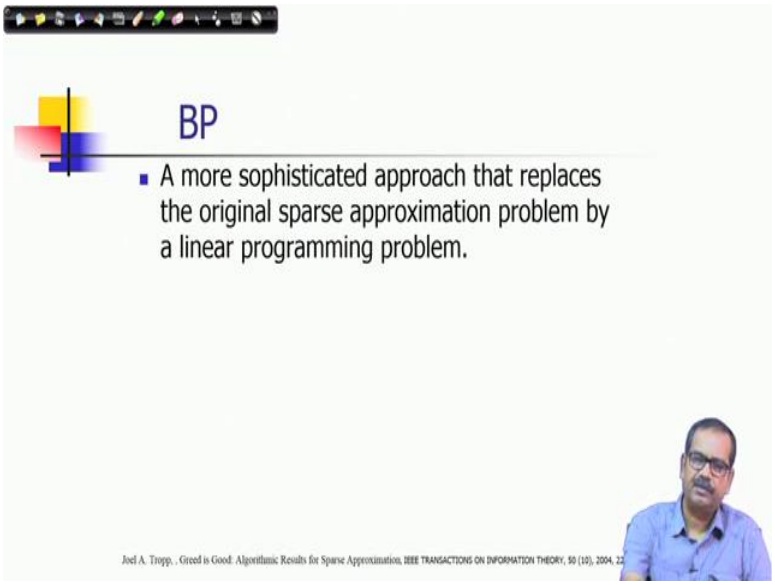
There next time what can you subtract the represented part already what is being done by that vector that is called residual. The next time you are going to carry you in the same operations with residual, so with residual again you find out along which directions you are getting maximum component.

Then you add it to your dictionary use it for least squared error estimation, use it you as I mentioned that if I give you a set of basis vectors I can easily derive what is the best representation best linear combinations which will represent the corresponding vector. So, use it use that technique to get derive the best representation and continue doing it for m atoms m number of times than or as long as you want to represent the data.

So, it produces a new approximate by projecting the residual onto the dictionary elements that have already been selected, what I described that is mentioned. Here that every time you are considering residual part of the input vector you project it on a particular vector a dictionary element which has not has been used already.

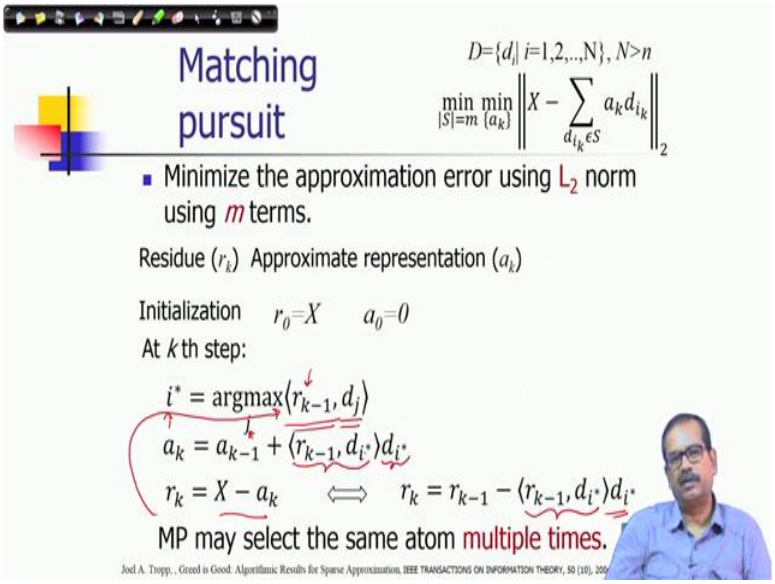
And the dictionary element which is giving you the maximum value of projection that should be included into your set S. And use set S to represent the efficient representation it extends the trivial greedy algorithm that success for an orthonormal system that is a characteristics.

(Refer Slide Time: 14:14)



The other approach which is called basis for suit it is a more sophisticated approach. And what it does it replaces the sparse approximation problem as a linear programming problem I will explain elaborate that problem statement.

(Refer Slide Time: 14:29)



So, before describing the orthogonal matching pursuit algorithm. Let me describe a similar algorithm which was say it before orthogonal matching pursuit it is simple matching pursuit algorithm. So, the idea is here that at every time you just look at only that those you know elements of the dictionary where you get the maximum residual component; and use that you know element itself to represent the vector.

So, it is minimize the approximation error using L 2 norm using m terms that is objective. And you are considering that at k th iteration since it is iterative process, you have residue representation as $r_k = X - a_k$.

So, the initialization is that your whole signal or whole vector input vector itself as X like a residue r naught is equal to X and so far you do not have any approximation which is 0 which you need to do. Then at every step every k th step what we are doing, as I mentioned you are finding out the direction finding out that element of dictionary where you get the maximum you know projected component.

So, this is the residual at this stage, so what is the projected residue along this particular you know dictionary element and then consider only that dictionary element where this value is maximum. So, you choose that dictionary elements say i star is that index, so now use it for representing this signal. So, this would be your magnitude and this would be your direction of the vector.

So, now you are, so the signal which is represented by a k that is the approximation approximately representation at the kth iteration, it should be incrementally it is representing that approximation. So, with previous approximate representation you are adding this vector as I mentioned. So, this is the magnitude part of that vector which is the component what is there in the residue. And this is the direction of the vector, so d_i* gives you the vector itself.

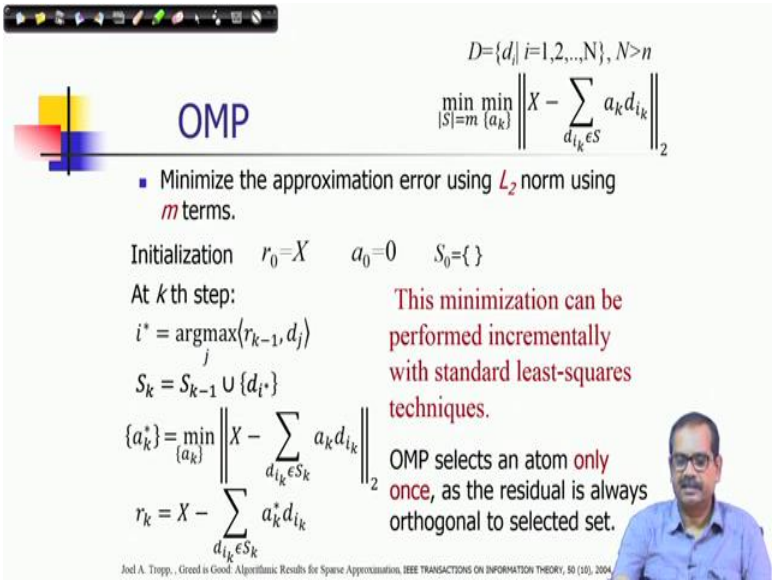
So, you add it with a k - 1 that is previous approximation and then you get the corresponding approximately representation. So, now, the residue rk, ak th rate step would be this and this residue will be for the next iteration use this residue once again for finding out the next of it is representation approximations. So, you privet this process till you get a good approximation and you should note that this is equivalent to this competition.

So, I can subtract the approximations what is the approximate representative signal with from the total signal then also I get the residue at this stage or I can incrementally build up the residue. So, earlier I had residues still k - 1 stage and out of that already this part is accounted. That means, the rk-1 this residue the projected component of the residue as in di* it has been already taken care of it has been transferred to the approximation part.

So, the residual part gets reduced, so slowly residue is reducing and it is trending to 0. So, when it becomes 0 it becomes exact representation otherwise, the whatever part is there that is the approximation still that error is error representation will remain as a residues. Now, problem of matching pursuit as you can see in this process it may select the same dictionary element again and again.

So, we have not put any restriction we are concerting of the throughout the whole dictionary element itself. So, if you are a residue it is still, so at some stage once again you can get the same dictionary element and if I am trying to restrict per m terms. So, I have to also keep a separate count that how many elements are presenting taken care of in my representation of this signal.

(Refer Slide Time: 19:02)



In orthogonal matching pursuit algorithm we take care of that factor, so that the elements of the dictionary they do not appear repeatedly. So, what because we are we will be performing here least square estimation, so your residue will be always orthogonal to the selected set of dictionary atoms. So, that is the property we can ensure by doing least square error estimation. So, here the process goes like this first you have to initialize it, so you have this two atoms r_0 =X and a_0=0.

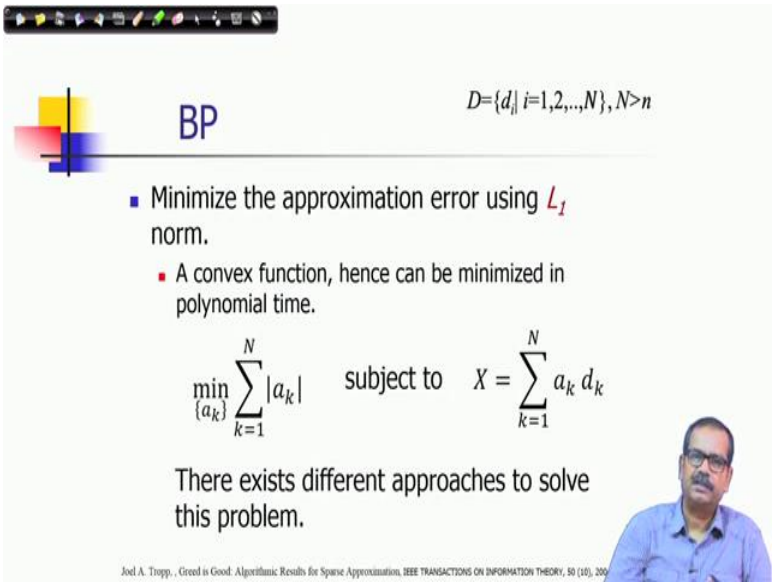
So, this is the same initializations and at kth step and also you are considering the set which will be including the set of dictionary elements which will be reconstructing the signal. So, at kth step first we find out that what is the direction along which you get the maximum component of the residue. And in the same way what we did for the matching pursuit and once you have chosen that then you put it into the set of atoms what would we use for representing the signal X or the input vector X.

And we would like to find out the best linear approximation there we will be using the least square error estimate method. So, that is the difference from matching pursuit, but by doing it what you are ensuring that all the atoms selected in the set S. That means, Sk at k th iteration all these atoms they are orthogonal to the residue rk at that stage, k th stage residue. So, the rescue is computed like this, so I mentioned that this minimization can be performed with standard least square techniques.

So, this is a residue and this residue it we have ensured through the step by enforcing least square error estimation that rk is orthogonal to all the elements of Sk. And that is why in the next iteration you do not require to bother about finding out the elements.

You are only choosing those elements which are remaining elements of dictionary which are not within the Sk. And you go on doing these things till m th terms for best m terms approximation this is what is orthogonal matching pursuit. So, as I mentioned that OMP selects and atom only once as a residual is always orthogonal to the selected set.

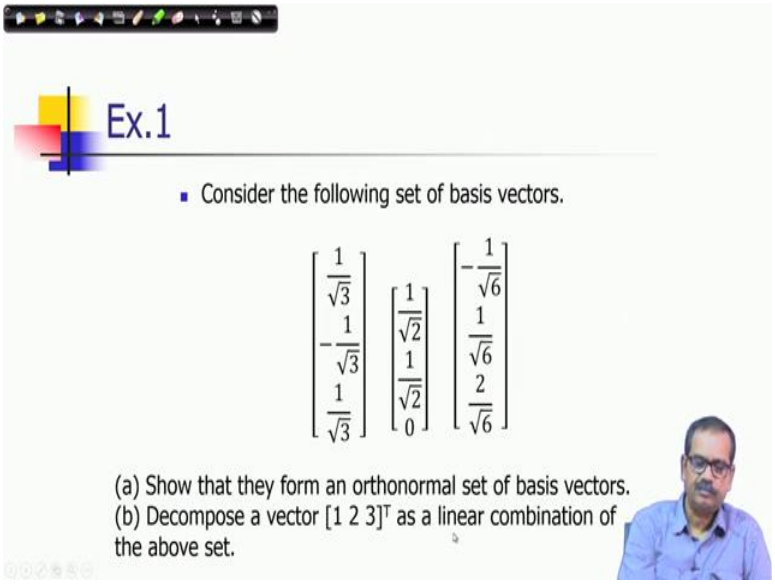
(Refer Slide Time: 22:03)



Basis pursuit problem is different form orthogonal matching pursuit problem. It minimizes approximation error using L1 norm and that too it is L1 norm on the coefficients of representation; that means, coefficients of the linear combination. And it gives you a convex function and hence it can be minimized in polynomial time and you can see that this is the problem statement as I mentioned that you are trying to minimize the sum of coefficients represent the signal.

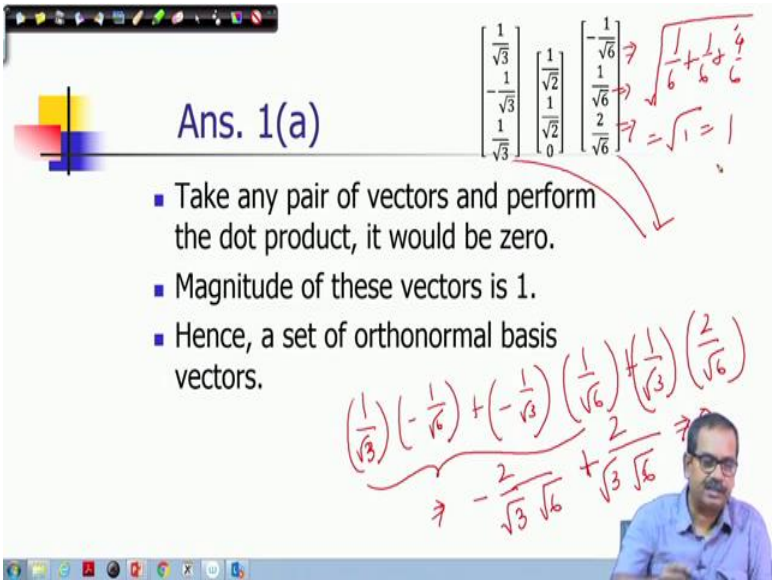
And by that you will be achieving as sparse representation and there exists no different approaches. To solve this problem we are not discussing going to discuss any such solution it involves also mathematical complex mathematics to argue on those solutions. So, in this particular case we will be considering only orthogonal matching pursuit for representing for getting a sparse approximate representation of a signal using a dictionary.

(Refer Slide Time: 23:13)



So, let me solve some exercises to elaborate this competitions, let us consider these problem that you consider the set of basis vectors there are three set of basis vectors. And you have to show that they form an orthonormal set of basis vectors and then if I given you these orthonormal set of basis vectors you represent a vector 1, 2, 3 as a linear combination of the above set.

(Refer Slide Time: 23:46)



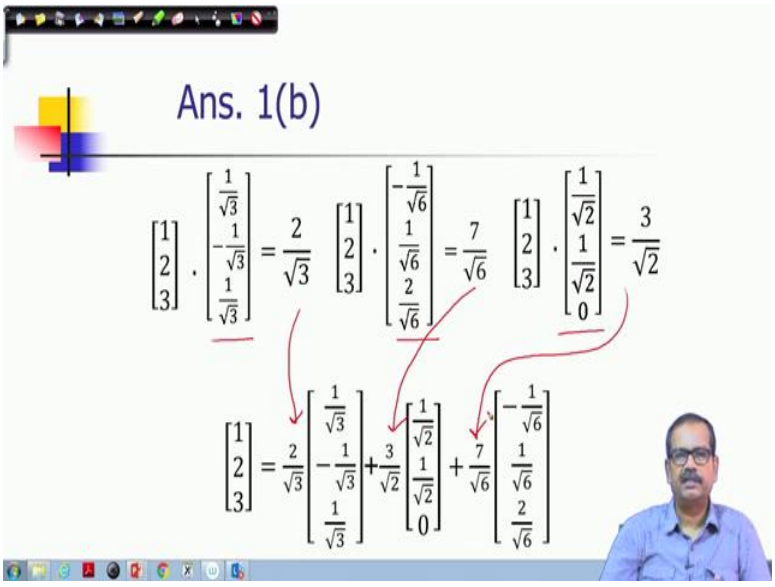
Now, for checking orthonormal setup basis vectors what you can do, you take any pair of vectors and if you perform dot product then it should be 0. So, any pair of vectors different

vectors and magnitudes of these vectors all the magnitudes of these vectors should be equal to 1. So, and that is how we can test and you can see here also just typically let me give you one example, suppose I have taken this vector and say this vector.

So, the dot product of this vector is 1 by root 3 into minus 1 by root 6 plus minus 1 by root 3 into 1 by root 6 plus 1 by root 3 into 2 by root 6. So, you will see that these are the two negative terms it should be minus 2 by root 3 into root 6 plus this is 2 by root 3 into root 6, so they should they should give 0.

And in this way you can choose any pair of vectors and you can find out they are dot product is 0 and if I take the magnitude, so take the magnitude of any of these vector say magnitude of this. So, magnitude means the square of minus 1 by root 6 which is 1 by 6 plus square of this 1 by 6 plus square of this which is 4 by 6. And you take the root of that and which is root of one that is it, so this is how you know you can show that this is orthonormal this is vectors.

(Refer Slide Time: 25:52)



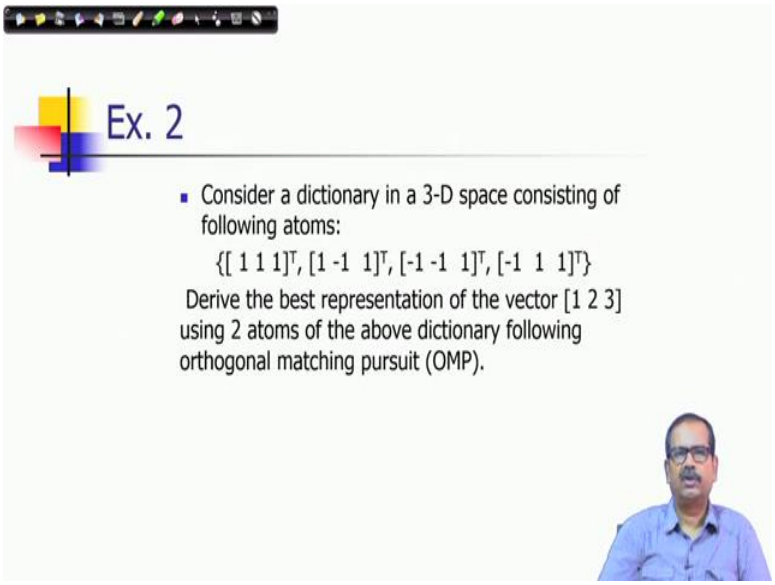
Considered the other example other exercise ; that means, how to represent a any arbitrary vector a vector 1, 2, 3 as a component. So, what we are doing you are taking the component of this vector 1, 2, 3 alone all these orthonormal basis vectors another basis vectors.

So, this is the component 2 by root 3 this is the magnitude 2 by root 3 along these vectors and 7 by root 6 along these vectors, 3 by root 2 along these vectors. So, finally, linear

combination can be expressed as you can see that this provides you the coefficients of linear combinations. All these dot products they are providing the coefficients of this linear combinations, we have discussed this theory in my lecture in image transform, so you can revisit that lecture once again.

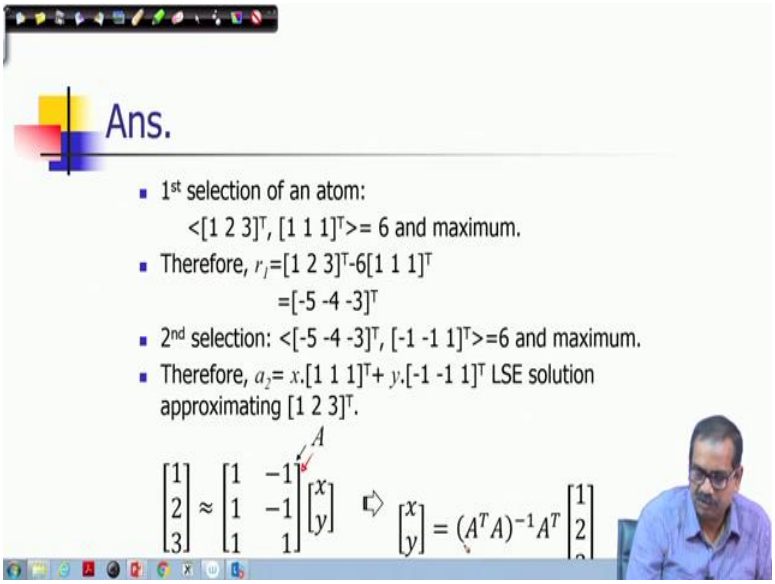
So, here the point is that if I give you an orthonormal set of basis vectors, then you do not require to do any complex operations what we discussed for matching pursuits. Simply you take the dot products with respect to any arbitrary vector and that value itself will give you the coefficients of the linear combination.

(Refer Slide Time: 27:11)



Let me take the other example, if I consider a dictionary in a 3 D space consisting of this atoms, so you have seen there are four atoms there in the dictionary of 3 d space and we can show that they are all linearly independent atoms. So, which means it is, so over complete set of dictionary as you know in 3 D space, 3 linearly independent vectors are sufficient to represent any arbitrary vector. So, derive the best representation of the vector 1, 2, 3 using two atoms of the above dictionary following orthogonal matching pursuit that is what our no problem statement is.

(Refer Slide Time: 27:49)



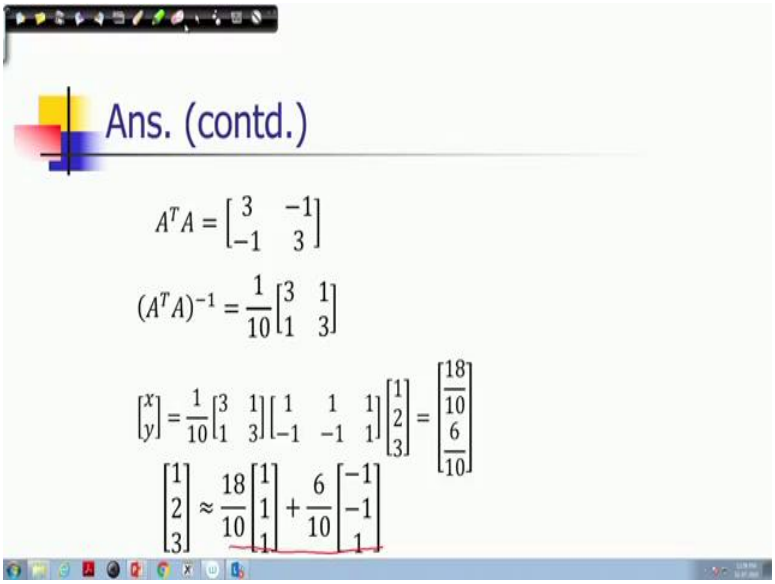
So, we will apply the orthogonal matching pursuit algorithm, first selection of an atom. So, find out the atom along which you get the maximum you know dot product value or component value. So, you take this dot product of these 2 and you get 6 and that is the maximum, you can check over other 3 atoms also it is a maximum. So, you select this atom for the representation which means this is the magnitude and the direction is 1, 1, 1 there is a vector.

So, the residue would be 1, 2, 3 minus 6 1, 1, 1 transpose I am for the convenience of representing in one slide I am using row vectors representation of this you know particular vectors. So, your residue at this stage becomes this vector, now the second selection. So, the second selection means now which atom from the remaining set of atoms one second you perform the dot products or with each of them.

And you can observe that minus 1 minus 1 1 that is giving you the maximum value of these dot products and that value is also 6 here. So, now, you have this in your dictionary you have to atoms 1, 1, 1 and minus 1 minus 1 1. So, use this a dictionary to obtain the best linear combination representation of the vector and you can use as you can what you can do that you from that best set of this is vector is B as we have done.

So, this is what is B and this is what is A in our representation and this is input this is what was X in our representation. And then oh here actually there is a some confusion of this, we are we are considering these values A these matrix is A here instead of N.

(Refer Slide Time: 30:06)



So, now, you get this is a solution, so to compute that A transpose A inverse you can compute. And you can compute the linear combination those coefficients X and Y using that this pseudo inverse by using pseudo inverse you can compute it. So, finally, your linear combination of the dictionary elements the best linear combination using two atoms for 1, 2, 3 is given this, so this is your required solution these the linear combinations that you are getting. So, with this let me stop at this point and we will continue this discussion for sparse representation in the next lecture.

Thank you very much for listening.

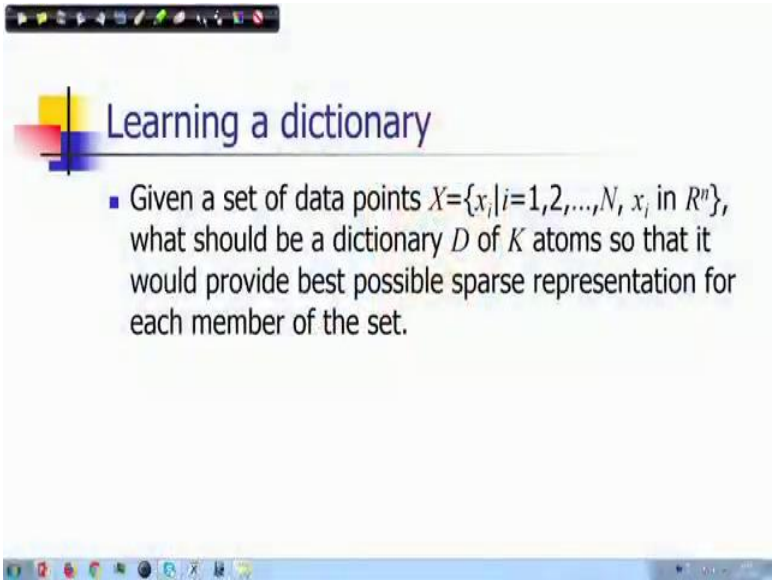
Keywords: Sparse approximation, orthogonal matching pursuit, basic pursuit.

Computer Vision
Prof. Jayanta Mukhopadhyay
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

Lecture - 54
Dimension Reduction and Sparse Representation Part – IV

We are discussing about Sparse Representation of signal or input data vector. And in the last lecture we have discussed given a dictionary how we can obtain sparse representation of a signal using different pursuit algorithms particularly we have discussed about orthogonal matching pursuit algorithms.

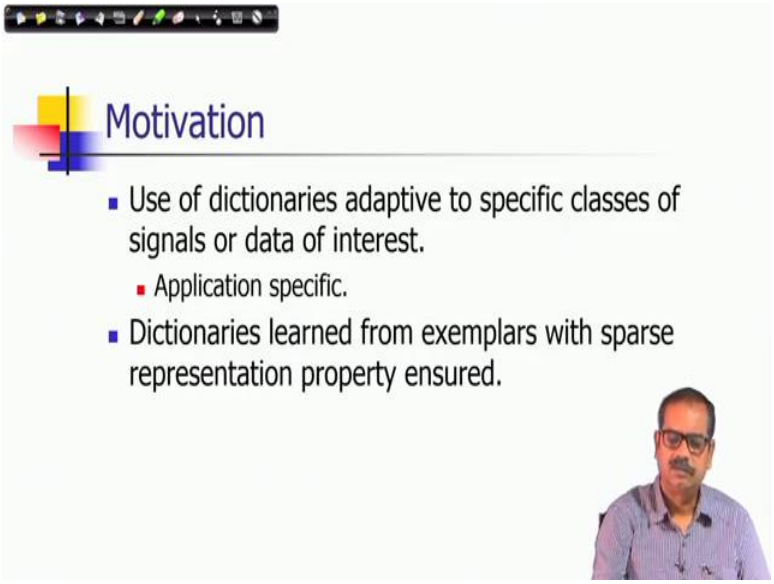
(Refer Slide Time: 00:31)



Today I will discuss about another problem of sparse representation, here we would like to learn the dictionary itself. So, in the previous case we have considered the dictionary is given and given the dictionary of say K atoms we wanted to get a sparse representation given any input signal of dimension n.

Now, in this case the problem is that we would like to learn a dictionary specific to a set of data. So, given a set of data points here we have considered the symbol X as a set and xi which is representing n dimensional data vector. Suppose you have N number of such data vectors in that set, then what should be a dictionary D of K atoms so that it would provide best possible sparse representation for each member of the set.

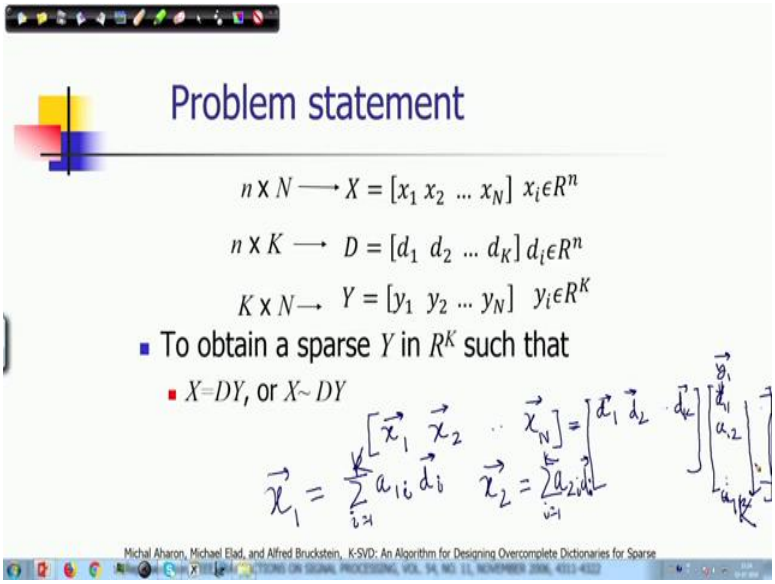
(Refer Slide Time: 01:41)



Motivation of learning dictionary could be several that could be several such motivations. In particular you would like to learn dictionary adaptive to specific classes of signals or data of interest. So, that would make a dictionary suitable for certain application and would give you relatively better performance compared to any standard fix dictionary. And, this dictionary learns from exemplars and it also ensures that sparse representation properties are insured using this dictionaries.

So, sometimes it may happen that if we use a fixed dictionary given any input data representation sparse representation may not be possible with that set of atoms in the dictionary. But, if we can tune the dictionary for a specific classes of data set then most likely we will be getting a better sparse representation of data. So, this is also another motivation.

(Refer Slide Time: 02:58)



So, just precisely let us define the problem statement here for this dictionary learning, we consider a data as an n dimensional vector and you consider a set X which is which consists of N number of data points and let us represent it in a matrix form. So, we can consider a matrix X by each column represent the data and each column is of N dimension.

And, then we are considering there are dictionaries of K atoms and again each atom is of N dimension whereas, it has to be the same as the dimension of the data we would like to have linear combination of this atoms to represent the data. And, that linear combination should be sparse and the coefficient of linear combination is represented by vectors yi's.

So, Y is also another matrix where each column of Y represents the sparse representation of data X. So, the dimension of Y is K dimensional vector because K dimensional representation of data point in this case and so, the column vector of the matrix Y of dimension K. So, our problem is that we would like to obtain a sparse Y in the K dimensional real space such that X equals DY.

So, this expresses itself the linear combination of dictionary atoms for each data point and the coefficient of linear combinations they are coming from the columns of Y or it could be approximate. So, let us considered the let us make a dimensional check of this particular fact; that means, we are trying to factorize the input matrix data matrix X into two factors. One part should give me a dictionary of K atoms, the other part should give me a dictionary of N sparse representation for N data points; n here is N.

So, we consider the dimension of X this is say n X N, then dimension of D this is n X K as we can see that n is the size of the column vectors. So, that is why that many rows should be there that is n and K is the number of atoms in dictionary D. So, that is why its number of columns should be K columns. Similarly for Xn should be the size of the row because n represents the dimension of the data point and N is the number of data points in X.

So, number of columns in X is N. Similarly for Y it should be K X N. So, K once again is the dimension of sparse representation of data; that means, a few of those elements in that column vector Y should be nonzero rest of them should be 0 that is sparsity that is a interpretation of sparsity very few of them should be nonzero.

But the dimension could be very large, it could be quite large from n and that is a dimension of data original dimension of data and then N is the number of sparse represented data vectors. So, you can see the multiplication DY means n X K matrix multiplied by K X N matrix you get the dimensional matching here because you get n X N data matrix.

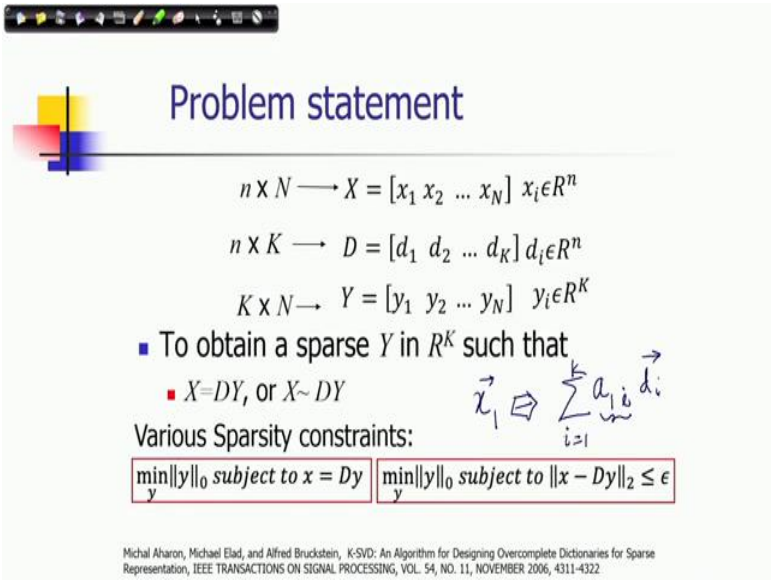
And let us also understand that how this representation is to be interpreted. So, we are considering that the data vectors [x1, x2,..,xN] and say the dictionaries are [d1, d2, to dK] and then each y1 let me write y1 as see a11, a12 to a1K and like this. So, if I consider only the first row, if I consider if I consider this dictionary and these are the vectors these are the atoms and you know these are the this is the coefficients.

So, x 1 vector and this is y 1 vector, but which is represented by this K dimensional vector. So,

$$\vec{x}_1 = \sum_{i=1}^{K} a_{1i}\vec{d}_1 = 1$$

So, in this way for all n vectors you have one column at y; one column of Y. And, this is how these vectors are represented and this is how these linear combinations of each column vector of x is represented by these corresponding matrix multiplication of D and Y.

(Refer Slide Time: 10:10)

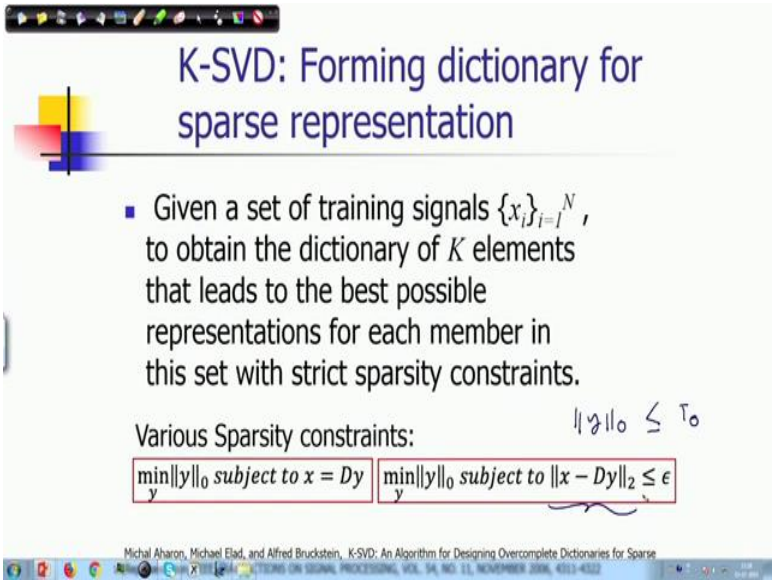


So, you can apply various sparsity constraints as I mentioned that the K dimensional representation of input vector which are given in the matrix Y now these would be sparse which means there should be only a few nonzero elements. So, when I have written a representation say x 1 this vector when I am representing y say some of $a_{1i}\vec{d}_1 = 1$ to k only few of this coefficient should be nonzero and rest of them should be 0.

So, you can apply various sparsity constraints there. So, one of the some of these examples of these constraints can be shown here say the 0th norm of y; 0th norm means number of nonzero element number of nonzero element of y and that should be minimum. You should have that representation that your representation of dictionary atoms and also these sparse representation would be such that the number of zero nonzero elements of the each sparse represented vector that number should be kept as minimum.

So, if I consider the whole set y al the set y you can count how many number of nonzero elements are there in that matrix that can be also consider as we measured and we would like to minimize it. Or we can consider that the approximation; that means, it is for exact reconstruction X = DY we can consider this or it may be approximate reconstruction that it could be you may not get exact reconstruction of all the input vectors that could be a tolerance of Epsilon in the reconstruction in the sense of L2 now and there also it would like to minimize the 0th now of the Y vectors.

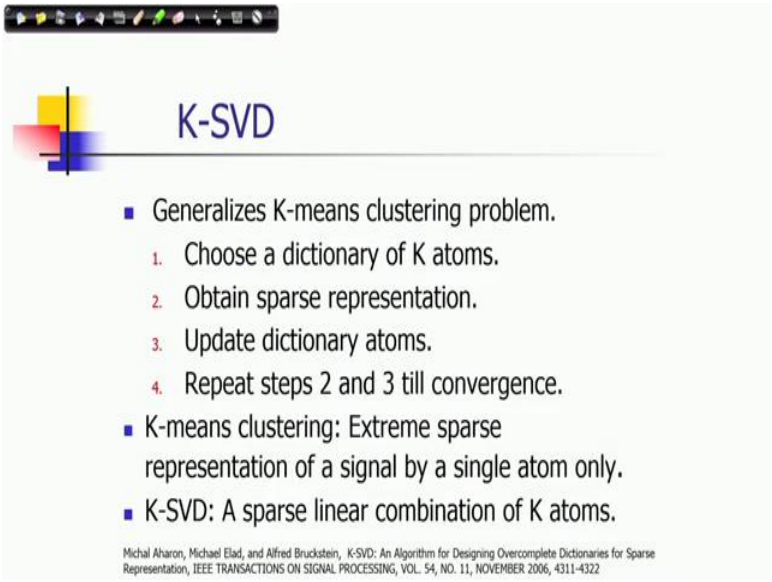
(Refer Slide Time: 12:30)



We will discuss if particular type of dictionary forming algorithm which is called K- SVD or K singular value decomposition and in this case it is a same problem statement as you can see it is the given a set of training signals to obtain the dictionary of K elements that leads to the best possible representation for each member in this set with strict sparsity constraints.

Now, the sparsity constraints here we have mention with this I would like to add in this case instead of considering always minimization we can make a bound that the 0th norm it should be less than or equals sum constant T0 which is a small number. And you would like to minimize the L 2 norm in this case that we will took by keeping this constraints we would like to minimize the L 2 norm of the reconstruction.

(Refer Slide Time: 13:44)

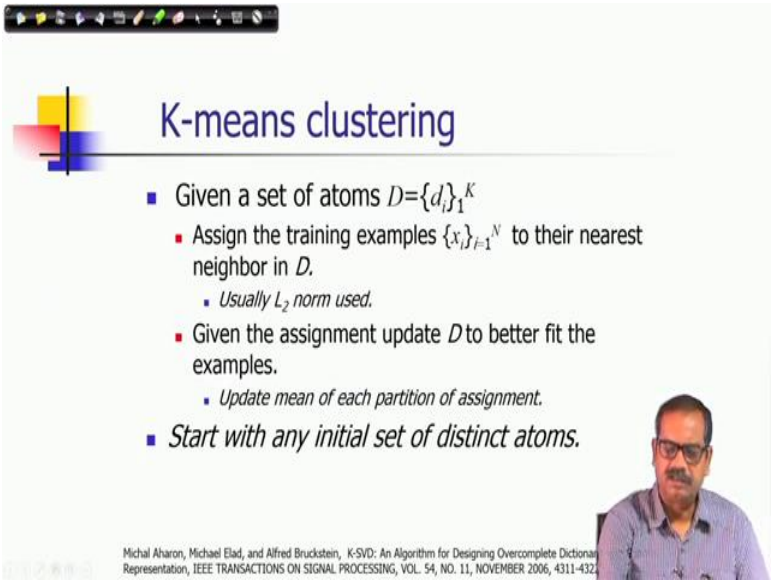


So, the principal of this algorithm K similar value decomposition or K SVD that is how this algorithm is popular to popularly known, it generalizes the K means clustering algorithm or K means clustering problem rather. And, we know that in K means clustering what we get, we get representatives of K groups of data vectors and each representative acts like an atom in this context. And then you get an extreme sparse representation if I consider the any member of that group is represented by that atom only.

So, we will see that in that in terms of Y matrix how this vector looks when we consider its an extreme sparse representation. So, in the case of and in K SVD instead of only a single atom representation of K means clustering we consider a sparse linear combination of K atoms. So, what it does it chooses a dictionary of K atoms and then it obtains a sparse representation and then it updates dictionary atoms to get a better representation out of this and repeat steps two and three till convergence.

So, once it obtains a sparse representation using this atoms then it checks whether any sparse representation is possible by updating some of destruction atoms and also with better approximation of the input signals. And, if it is possible then it updates and it repeats its processes till there is no such improvement is possible.

(Refer Slide Time: 15:41)

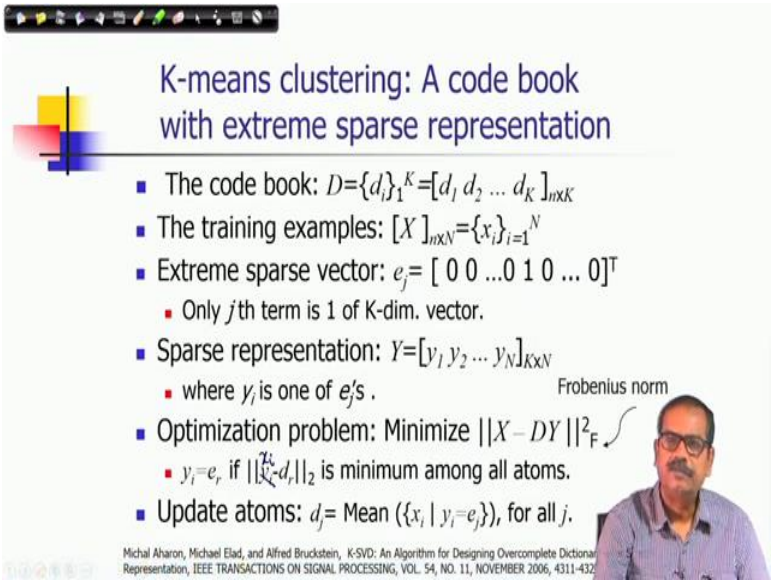


So, let us revisit this K means clustering algorithm and width in the context of this particular discussion. Say you have a set of atoms which from a dictionary you remember that we have to initialize the centers of clusters K centers of clusters now those initial centers are considered here as atoms of dictionary.

And then what we do that we assign the training examples to their nearest neighbor in the dictionary D. So, that is what we did in K means and then given an assignment then we update D to better fit the examples. So, far K means clustering we have used L 2 norm to check to compute the distance between any sample vector to the dictionary to the atoms or to the centre of clusters.

And then whichever is nearest we assign that vector input vector to that cluster and once the assignment is over then you have once again K partitions, then again re compute the centers. So, that is what the update is updation process see update mean of each partition of assignment. We started with any initial set of distinct atoms and then after convergence you consider those are the learn dictionary atoms.

(Refer Slide Time: 17:13)



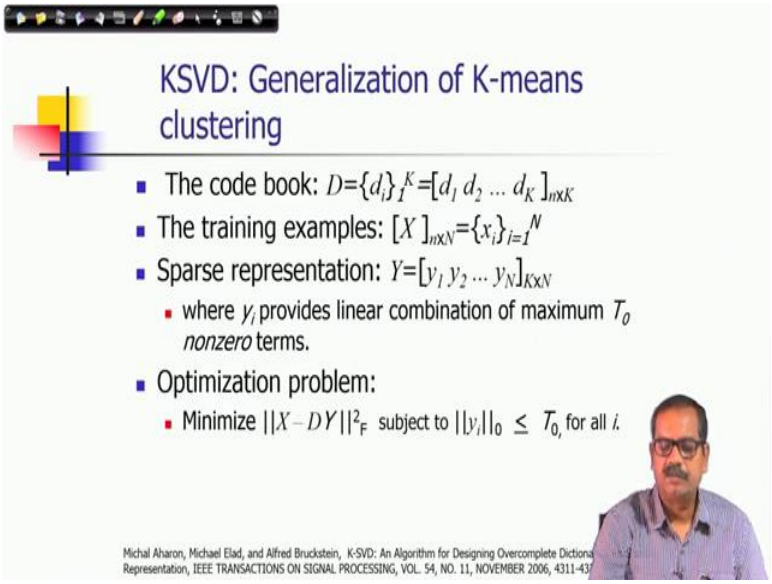
So, your codebook here is the K cluster centers those are the dictionary atoms that is your code book and you have this training examples and this is what is the representation exchange sparse represent sparse vector you can say j ith vector representation would be in this form all the elements would be 0 except 1. So, the 1 denotes that atom which is assigned to that input vector or this input vector belongs to that group where this atom is the centre of that cluster.

So, that is why it is called extreme sparse because it has only one nonzero element and rest of them at 0. So, in your sparse representation you have only this type of vectors; that means, extreme sparse vectors it should be one of them and your optimization problem as we have considered for K means. It is once again as you can see your minimizing the L 2 norm square of the L2 and your update policies that if ith input vector is closest to the dictionary atom r th dictionary atom then assign the r th sparse r th you know, then the presentation should be here which is exchange sparse vector which means an nonzero elements 1 would be only at the r th location.

So, once you have computed that location then for all the vectors once the assignment is over, then you update each partition which means that you are considering mean of those input vectors which have the same assignment. For example, with the same assignment of e j that is the sparse vector representation for the j th group and you compute its mean that would be your new dictionary updated dictionary atom.

So, this operation you are doing iteratively and at the when it converges then we are you stop and that is the dictionary you learned and already the sparse representations are also derived as extreme sparse vectors for each input vector. So, you have mentioned this is problem provenience norm that you should consider.

(Refer Slide Time: 19:46)

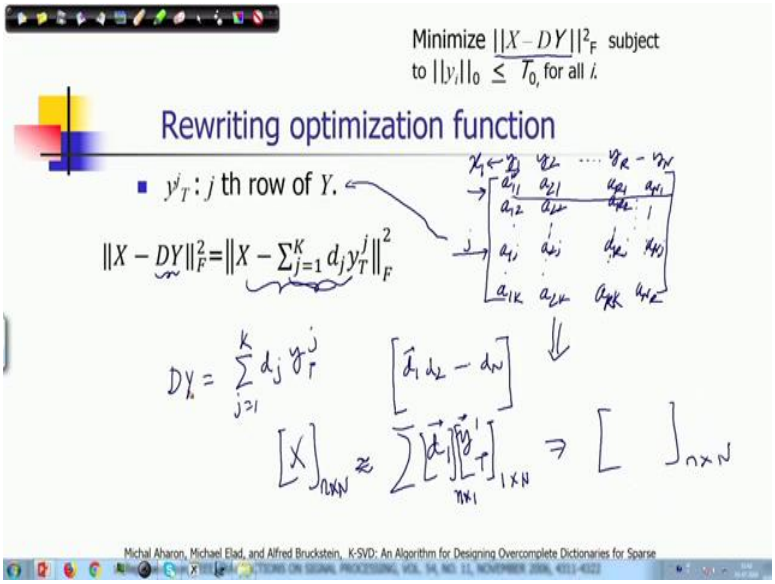


So, that was K means clustering and let us considered now that how this concept is generalized for K singular value decomposition approach or K SVD algorithm. In this case also we start with any initial codebook; we will discuss how this code book is derived here and suppose there is an initial course code book.

And then and also you have the training examples as given in this form a N number of data points and it can be represented as a data matrix X and you would like to get sparse representation once again a matrix of K X N each column represents the sparse representation of K data vector. And why I provide linear combination of maximum T naught nonzero term that I mentioned while discussing this sparsity constraints that is involved in KSVD algorithm.

So, the optimization problem here is that you would like to minimize the provenience norm X - DY its a generation of L2 norm square and subject to that number of nonzero elements for each sparse representation is less or equal to T 0.

(Refer Slide Time: 21:14)



So, this computation is carried out by a nice manipulation of the optimization function that we will discuss here. If I consider my optimization function if you note here the optimization function is given this; that means, provenience norm of X - DY and how this can be written in a different from that is that we would like to discuss now consider the j th row of Y. So, if you remember the representation of y each column is a sparse representation. So, I have represented by the coefficients a 1 1, a 1 2 to see a 1 k.

So, this is the y 1 of the sparse represented vector and this is representing x 1. So, y 1 is represented representing x 1. Similarly y 2 can be represented as linear combination say a 21 a 22, so this is my say notation a 2 k. So, this is y 2 and in this way you are representing say any particular small k y as see a k1 a k2 to a k1 is k another is K; K is the dimension and finally, there are n number of representations because there are n number of data points.

Now, you consider any particular row say j th row which means this should be a 1j a 2 j a small k j a capital N j. So, this j is represented as the is j th row vector. So, now, this expression X - DY this can be conveniently written in this form you can see that what is doing it is the sum of the rank 1 matrixes in the other way.

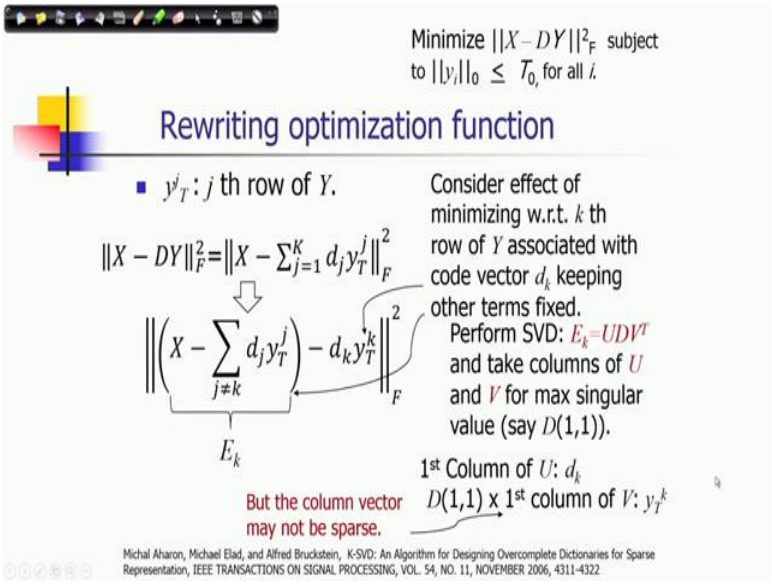
So, you have if I write it in this way that d 1 d 2 to dn and this is the y matrix. So, so we can write it as say d 1 into this first row; that means, y 1 T in our notation. So, d 1 this into this. So, the dimension of d 1 is n X 1 and y 1 T its dimension is 1 X N because there are

N elements. So, there are N elements in a rho there are N columns. So, and only 1. So, this would give you a matrix of n cross N and x is a matrix which is of n cross N.

So, if I consider sum of all these matrixes that would actually cover the x. So, it is a same. So, this DY; DY is the same as this 1 it is not same it is trying to represent that we are trying to get that D and Y which will be which will be the same as a x if I multiply them, but $DY = \sum_{j=1}^{K} d_j y_p^j$.

So, this is the trick. So, you are considering every atom dj and find out what is its contribution to the reconstruction of DY the whole multiplayer multiplied factor. So, the advantage here is that if you do that, then what you can consider you can separate out the contribution of a single atom say d k.

(Refer Slide Time: 26:05)



So, this is the single atom d k is the contribution of atom dk and this is a contribution of rest of the atoms towards formation of DY and which we would like to get as close as possible to x. So, if we have this then as I then the idea is that since we are separated out. So, we can observe just what the effect of a single atom is.

So, if I keep all other thing constant then what is the best dk and yTk that should approximate this particular factor let me call it as E. So, we can compute it because everything is given in your in your hand at this stage you have the input data vectors, you

have the set of atoms that is the safe suppose you are doing iteratively. So, K - 1 th set of atoms you have also the sparse representation in the previous K - 1 th sparse representation.

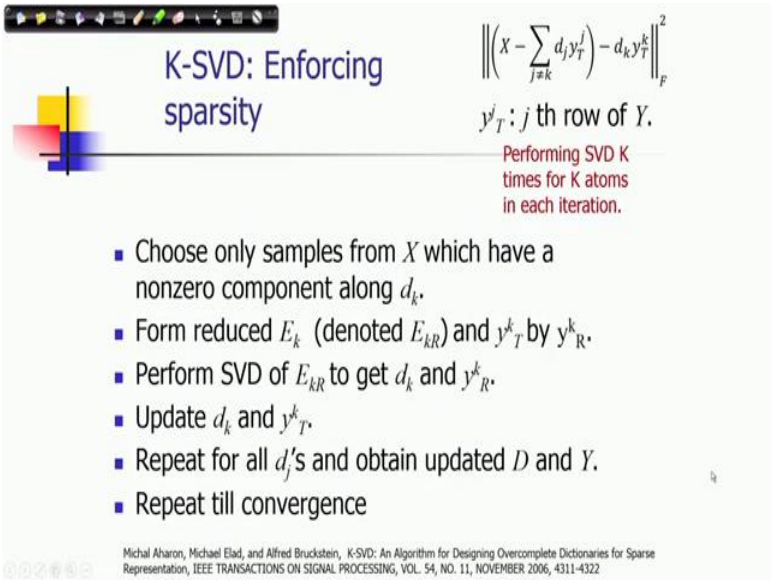
So, you can find out what is the value of this error of approximation using except K th atom this is an error approximation. So, the error can be this minus this. So, if I can update dk and y T k such that it is close to e then my error would be minimized. So, that is a idea and that is what this algorithm does that keeping other transfixed. So, it gets E k.

So, to do that what we can do? We can perform the SVD of E k. So, this is I am mentioning as E K and this is what is written. So, let me rub the my writing. So, if I perform SVD of E K and then you know that with singlar value decomposition you get set of or thermal vector column vectors of U and V and considered the columns of U the first column of u with the maximum singular value and column of b of the same maximum singular value.

So, you take the column of U and you can normalize it that is what is your d k and the maximum singular value into the column of V we will give you the y k T k that representation that could be 1 possible close approximation it is not exactly equal to E K. It is a best approximation of rank 1 approximation from the theory of approximation of matrix using this kind of singular value decomposition.

Now, this is one best representation, problem here this that here you are not ensuring that y T k of that coefficients would be sparse, so this is a problem that. So, as I mentioned we can consider 1st column of U as d k and the 1st column of V multiplied with the singular value d 1, 1 which is the maximum singlar value as y T k, but the column vector this column vector may not be sparse. So, what we should do in that case?

(Refer Slide Time: 29:28)

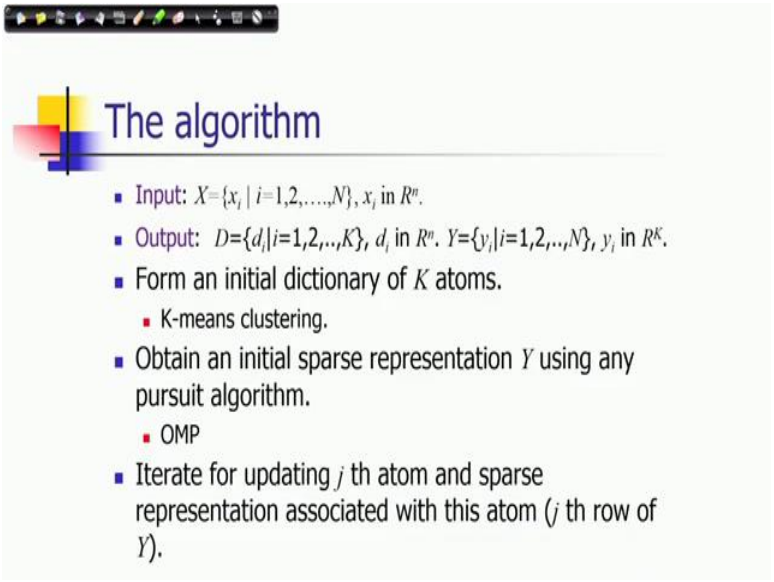


$$||\left(X-\sum \quad d_j y_T^j\right)-d_k y_T^k||^2$$

So, this is a tree that we can enforce sparsity by considering only those samples of the input which has nonzero contributions towards d k; that means, I have nonzero coefficient. So, choose only samples from x which have a nonzero component along d k and from there you can reduce the error matrix Ek to E kR. So, you are only considering those input samples related to that only and from there you can rewrite the equation. And, you can get E kR you can get the reduced representation of the row y R k and that you can use the once again singular value decomposition of E k R and then update d k and y T k from there.

So, you can repeat for all dj S and update obtain the updated d and y in this form and you should repeat till repeat this thing till convergence. Actually convergence like K means clustering here also the problem here that it can get local it can start at a local optimum point in this case. So, performing SVD since you are performing this singular value decomposition K times for K atoms in each iteration we call this algorithm as K SVD.

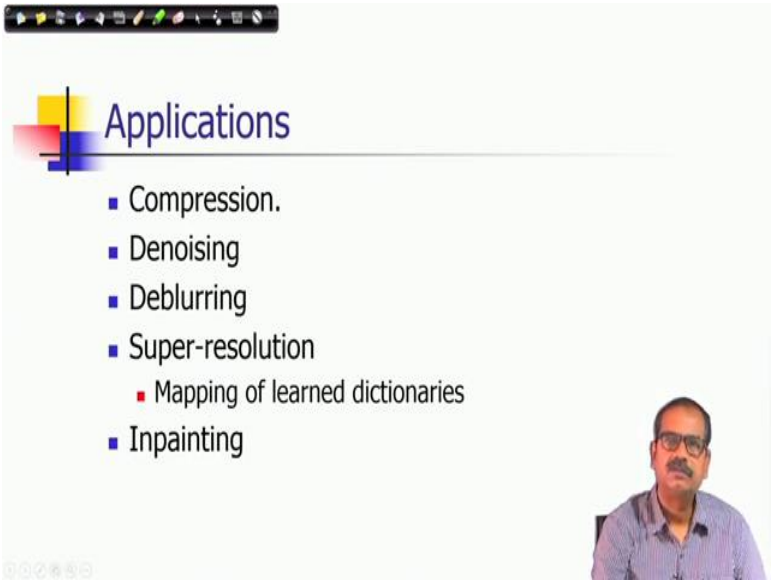
(Refer Slide Time: 30:53)



So, finally, if we like to give the overview of this algorithm from total perspective at your input is x data points sorry n capital N data points and represented by this set X or which can be also represented by a data matrix X and your output is your K number of atoms of the dictionary D and also the sparse representation of each input sample Y.

So, the first step is that you have to form an initial dictionary of K atoms and that any method you can use K means clustering can we also be used, then obtain an initial sparse representation y using any sparsity algorithm. So, you can use K means clustering for that or you can use orthogonal matching pursuit for this sparse representation, then you iterate for updating j th atom and sparse representation associated with this atom and you go on to in this iterations till there is a conventions.

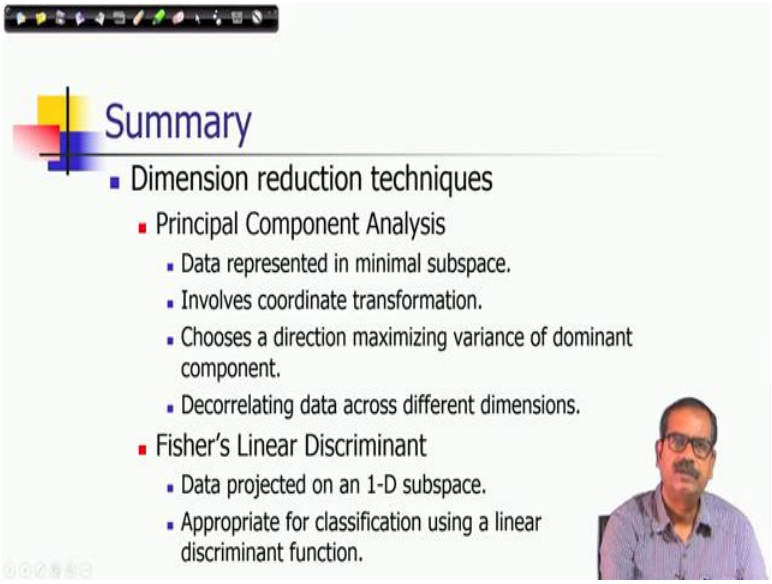
(Refer Slide Time: 31:59)



So, there are some applications of the K- SVD algorithms I mean it can be applied for data compression there are various image processing operations where denoising is applied. Actually the idea here is that since once again you have factorized the input data you only retain those factors which are important and where the coefficients are negligible then you can reject it. By doing this things you can compressed data, you can denoise data, you can deblur data. The other applications that kind of computation could be you can learn dictionaries of different levels of representation of a signal.

For example high level representation and low level representation and then if you establish the mapping between two dictionaries that it can give you an algorithm for increasing the resolution of signal. So, super resolution is that task which is called. So, mapping of learn dictionaries can be used in this case. In painting also another task of image processing where you take out some area and you feel that area by looking at the content of the image. So, that the discontinuities of extraction of the object from the image is not felt by everyone. So, they are also this kind of dictionary learning and mapping could be used.

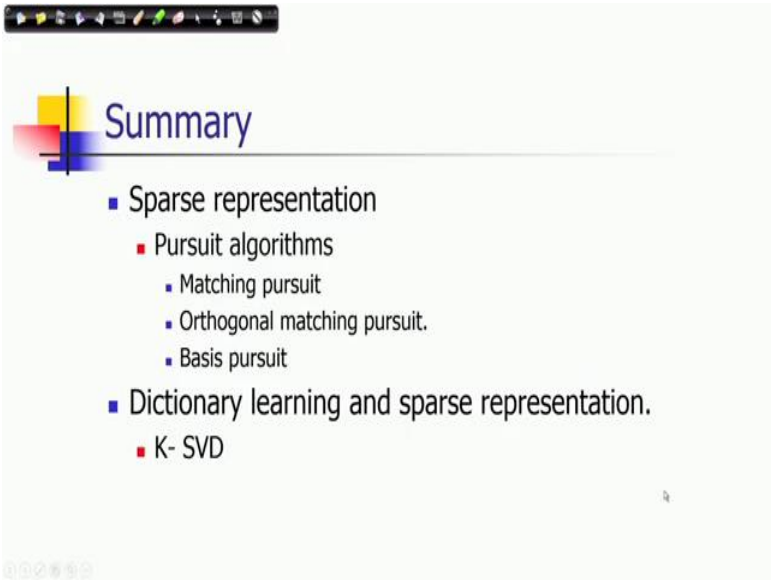
(Refer Slide Time: 33:23)



So, let me summarize the topics what we discussed under the title of dimension reduction and sparse representation. So, in dimension reduction we discussed about the technique of principal component analysis and also a technique Fisher's linear discriminant with respect to classification task.

So, far principal component analysis the objectives are to represent data in a minimal subspace and it involves we have seen it involves co ordinate transformation, it chooses a direction maximize in variance of dominance component and it decorrelates data across different dimensions. Whereas, in Fisher's linear discriminant the task was to data project the data in a 1-D subspace and then use it for classification and this projection itself gives severe linear discriminant function.

(Refer Slide Time: 34:19)



And, in spare representation we have discussed about different pursuit algorithms like matching pursuit, then orthogonal matching pursuit, then basis pursuit we have given just a problem statement, but we did not discuss the algorithmic steps. And, then we also discussed technique for dictionary learning and using it how to derive sparse representation in particular we discussed about this algorithm K- SVD. So, with this let me stop and this is the end of this particular topic we will start a new topic in my next lecture.

Thank you very much.

Keywords: K-SVD, K-means algorithm, denoising, deblurring.